

---

# **Quantitative Big Imaging - Dynamic experiments**

**Anders Kaestner**

**Apr 23, 2026**



## CONTENTS

|      |                                                             |    |
|------|-------------------------------------------------------------|----|
| 0.1  | Dynamic Experiments . . . . .                               | 1  |
| 0.2  | Imaging of dynamic experiments . . . . .                    | 3  |
| 0.3  | Motivation spatiotemporal analysis . . . . .                | 5  |
| 0.4  | Dynamic experiments . . . . .                               | 7  |
| 0.5  | Tracking objects and changes . . . . .                      | 17 |
| 0.6  | Key Points (or feature points) . . . . .                    | 26 |
| 0.7  | Tracking with Points . . . . .                              | 28 |
| 0.8  | Features and Descriptors . . . . .                          | 28 |
| 0.9  | Computing Average Flow . . . . .                            | 31 |
| 0.10 | Problems with tracking . . . . .                            | 38 |
| 0.11 | How to improve tracking . . . . .                           | 40 |
| 0.12 | Registration . . . . .                                      | 50 |
| 0.13 | Introducing Physics . . . . .                               | 58 |
| 0.14 | Two Point Correlation - Volcanic Rock . . . . .             | 60 |
| 0.15 | Summary . . . . .                                           | 60 |
| 0.16 | Visualization . . . . .                                     | 60 |
| 0.17 | Presenting the results - bringing out the message . . . . . | 61 |
| 0.18 | Summary . . . . .                                           | 73 |



This is the lecture notes for the 9th lecture of the Quantitative big imaging class given during the spring semester 2021 at ETH Zurich, Switzerland.

## 0.1 Dynamic Experiments

### 0.1.1 Load needed modules

```
import matplotlib.pyplot as plt
import seaborn as sns
from skimage.morphology import label
from skimage.measure import regionprops
import pandas as pd

from skimage.feature import corner_peaks, corner_harris, BRIEF
from skimage.transform import warp, AffineTransform
from skimage import data
from skimage.io import imread
from scipy.ndimage import distance_transform_edt
from skimage.filters import threshold_otsu

from matplotlib.patches import Rectangle
from matplotlib.patches import ConnectionPatch

import numpy as np
import pydot

%load_ext autoreload
%autoreload 2
plt.rcParams['font.family'] = ['sans-serif']
plt.rcParams['font.sans-serif'] = ['DejaVu Sans']
plt.style.use('default')
sns.set_style("whitegrid", {'axes.grid': False})

color_cycle = plt.rcParams['axes.prop_cycle'].by_key()['color']

%config InlineBackend.figure_format = 'retina'
```

### 0.1.2 Literature / Useful References

#### Books

- John C. Russ, “The Image Processing Handbook”, (Boca Raton, CRC Press)
- A. Ardeshtir Goshtasby, “Image Registration Principles, Tools and Methods (Springer Verlag)
- B. Jähne, “Spatio-Temporal Image Processing” , (Springer Verlag)

### Papers / Sites

#### Comparison of Tracking Methods in Biology

- Chenouard, N., Smal, I., de Chaumont, F., Maška, M., Sbalzarini, I. F., Gong, Y., ... Meijering, E. (2014). Objective comparison of particle tracking methods. *Nature Methods*, 11(3), 281–289. [doi](#)
- Maška, M., Ulman, V., Svoboda, D., Matula, P., Matula, P., Ederra, C., ... Ortiz-de-Solorzano, C. (2014). A benchmark for comparison of cell tracking algorithms. *Bioinformatics (Oxford, England)*, btu080–. [doi](#)

#### Keypoint and Corner Detection

- Lowe, D.G. Distinctive Image Features from Scale-Invariant Keypoints. *International Journal of Computer Vision* 60, 91–110 (2004). [doi](#)
- [OpenCV SIFT tutorial](#)

#### Registration

- [ITK registration guide](#)

#### Multiple Hypothesis Testing

- Coraluppi, S. & Carthel, C. Multi-stage multiple-hypothesis tracking. *J. Adv. Inf. Fusion* 6, 57–67 (2011).
- Chenouard, N., Bloch, I. & Olivo-Marin, J.-C. Multiple hypothesis tracking in microscopy images. in *Proc. IEEE Int. Symp. Biomed. Imaging* 1346–1349 (IEEE, 2009).

#### 0.1.3 Previously on QBI ...

- Image Enhancement
  - Highlighting the contrast of interest in images
  - Minimizing Noise
- Understanding image histograms
- Automatic Methods
- Component Labeling
- Single Shape Analysis
- Complicated Shapes
- Distribution Analysis
- Uncertainty and Statistics

### 0.1.4 Quantitative “Big” Imaging

The course has covered imaging enough and there have been a few quantitative metrics, but “big” has not really entered.

What does **big** mean?

- Not just / even large
- Being ready for *big data*
- Scalable, fast, and easy to customize

#### The three V's

##### Volume

The production of volume images can already be an obstacle for efficient data management and image analysis.

##### Variety

Scientific experiments need great variation to allow statistical analysis and hypothesis testing as we saw in last weeks lecture.

##### Velocity

Today's lecture will approach the problems arising when processes are observed. This means that time series of data is captured and these do by nature increase the amount of data to analyse and manage.

### 0.1.5 Outline

- Motivation (Why and How?)
- Scientific Goals

#### Experiments

- Simulations
- Experiment Design

## 0.2 Imaging of dynamic experiments

### 0.2.1 Dynamic processes

- Relating to objects in motion
- Characterized by continuous change, activity, or progress

### 0.2.2 Real time

- The actual time during which a process or event occurs.
- (Computing) of or relating to a system in which
  - input data is processed within milliseconds
  - so that it is available virtually immediately as feedback.

### 0.2.3 Imaging

Imaging is the

- capture,
- storage,
- manipulation,
- and display

of images.

### 0.2.4 Experiments

1. What sort of dynamic experiments do we have?
2. How can we design good dynamic experiments?



Fig. 1: Continuous processes evolve over time without guarantee that it returns to the same point.



Fig. 2: Repetitive processes always behave the same with a given frequency.



## 0.2.5 What information are you looking for?

We can say that it looks like, but many pieces of quantitative information are difficult to extract

- How fast is it going?
- How many particles are present?
- Are their sizes constant?
- Are some moving faster?
- Are they rearranging?

## 0.2.6 Image analysis

How can we...

1. track objects between points in time?
2. track shape?
3. track distribution?
4. track topology?
5. track voxels?
6. assess deformation and strain?
7. assess more general cases?

→ **How does this help answering your questions?**

## 0.3 Motivation spatiotemporal analysis

- 3D images are already difficult to interpret on their own
  - Obscured structures
  - Screens only show 2D
- 3D movies (4D) are almost impossible

### 0.3.1 2D movies can also be challenging

They are 3D!

- $x, y, t$

**Example: a water jet**

**Example: coffee making**

### 0.3.2 Other image series - 3D Reconstruction

#### Tomography

One of the most commonly used scans in the hospital is called a computed tomography scan.

This scan works by creating 2D X-ray projections in a number of different directions in order to determine what the 3D volume looks like

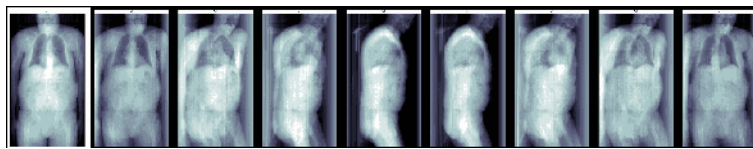


Fig. 1: Some views from a CT scan of a patient

#### Stage Tilting

Beyond just tracking we can take multiple frames of a still image and instead of looking for changes in the object, we can change the angle.

The pollen image below shows this for SEM

### 0.3.3 Scientific Goals of dynamic experiments

#### Rheology

Understanding the flow of liquids and mixtures is important for many processes

- blood movement in arteries, veins, and capillaries
- oil movement through porous rock
- air through dough when baking bread

#### Deformation

Deformation is similarly important since it plays a significant role in the following scenarios

- red blood cell lysis in artificial heart valves
- microfractures growing into stress fractures in bone
- toughening in certain wood types

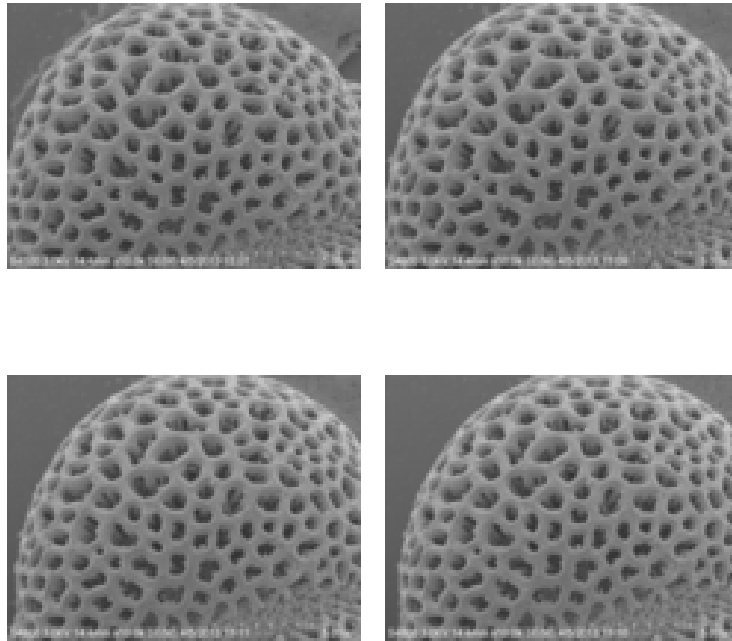


Fig. 2: A tilt series of a pollen grain.

## 0.4 Dynamic experiments

The first step of any of these analyses is proper experimental design.

Since there is always:

- A limited field of view
- A voxel size
- A maximum rate of measurements
- Dose limitations
  - Sample damage
  - Limited flux
- A non-zero cost for each measurement

There are always trade-offs to be made between

- getting the best possible high-resolution nanoscale dynamics
- and capturing the system level behavior.

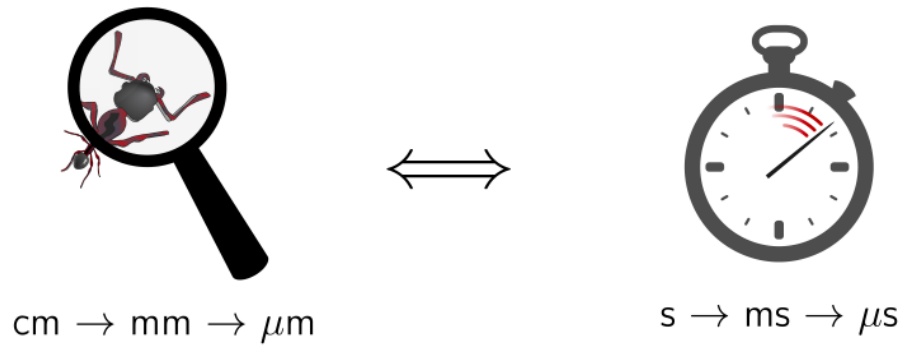


Fig. 1: The trade-off between time and resolution.

### 0.4.1 Factors affecting the image quality

- Process speed
- Spatial resolution
- Intensity dynamics



Fig. 2: Parts moving too fast will introduce motion blur.

### Example

We know from previous lectures that it is important to at least have a certain signal to noise ratio to be able to perform our analysis reliably. The SNR depends on the dose per pixel which can be controlled by several factors. In the following example we have the experiment conditions for an experiment. Now, if our preliminary assessment of the images tells us that the spatial *or* temporal resolution doesn't reach the requirements to allow quantitative analysis, then we have to change the frame rate or the pixel size. This can within some limitations be done without sacrificing the SNR.

You have a dose limited experiment

- 100 $\mu$ m pixels

- 1s Exposure time
- 800 neutrons per pixel

To maintain the SNR you have to

|              | Smaller pixels       | Higher frame-rate |
|--------------|----------------------|-------------------|
| Modification | Longer exposure time | Larger pixels     |

The reality is however that we have to settle with a compromise that often means a lower SNR. This compromise may then require new thinking for preprocessing and analysis. You may need different filtering and segmentation methods.

### 0.4.2 Rebinning

Rebinning or using larger pixels can be used to

- improve the SNR.
- reduce motion artefacts

Larger pixels can be obtained at experiment time by changing optical settings of the detector system or through binning or resampling of existing images after the experiment. The are pros and cons of each approach

- Setting pixel size at experiment time will optimizie the amount of produced data. Which has an impact on storage and processing if you produce long series. It is not possible to get higher resolution after experiment.
- Resampling has the advantage that it would be possible to go back higher resolution if needed. It will however produce more data the require additional processing.

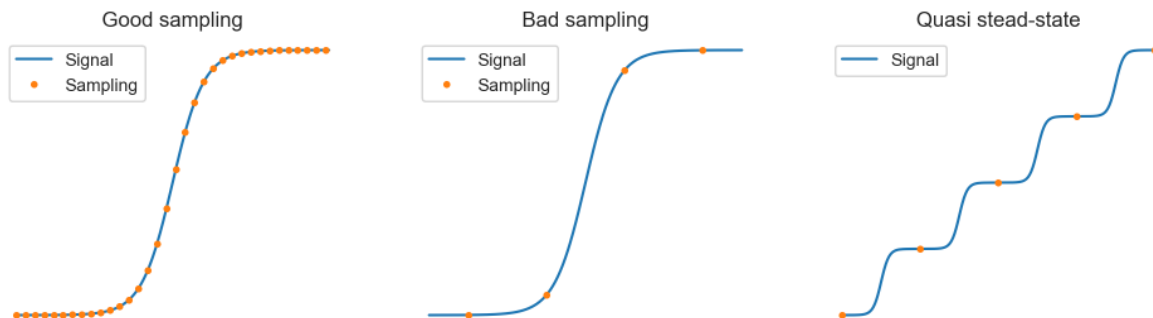
**...but you may miss small details**



Fig. 3: Rebinning improve SNR at the cost of small features.

### Frame rates

The sample rate needed to study a dynamic process must be chosen to allow several samples during the rise typically 5-20 times the fastest expected time constants in the system.



The first plot shows that an example where the rise of the signal is well covered by sampling points. This makes it possible to make accurate observations of the process. The middle plot is an example where the change in the process was too rapid and the transient information is lost.

In some cases it is possible to control the process and even stop the progress at specified steady state levels. This allows measurements of steady intermediate steady state conditions. This approach allows for longer acquisition duration while in the steady state.

### 0.4.3 Planning dynamic experiments

#### If we measure too fast

- sample damage
- miss out on long term changes
- have noisy data

#### Too slow

- miss small, rapid changes
- blurring and other motion artifacts

#### Too high resolution

- too few unique structures in field of view to track

## Too low resolution

- not sensitive to small changes

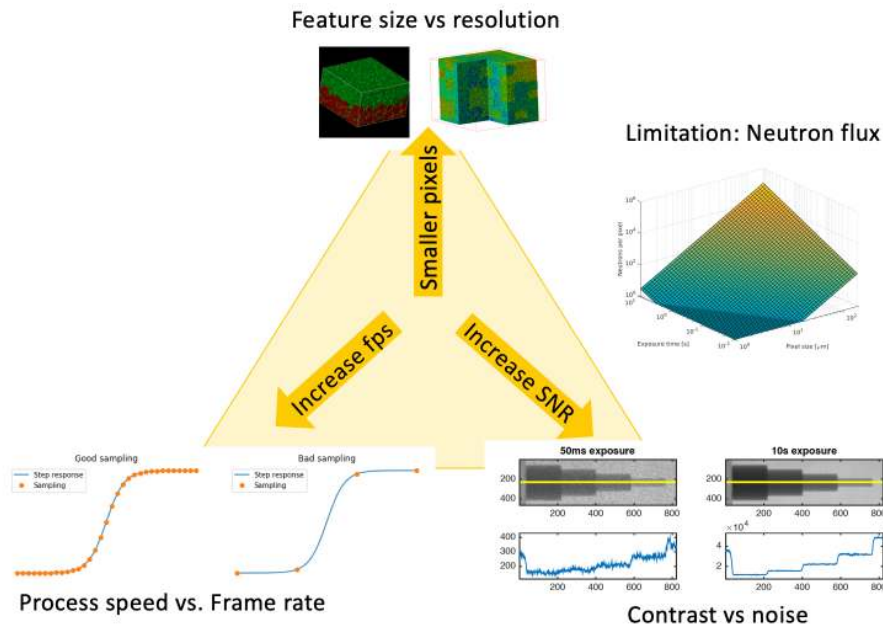


Fig. 4: Factors contributing to the deciding how to perform a dynamic experiment.

## Analysing trivial processes

For relatively simple processes you can extract images along the time axes. This will make the analysis easier.

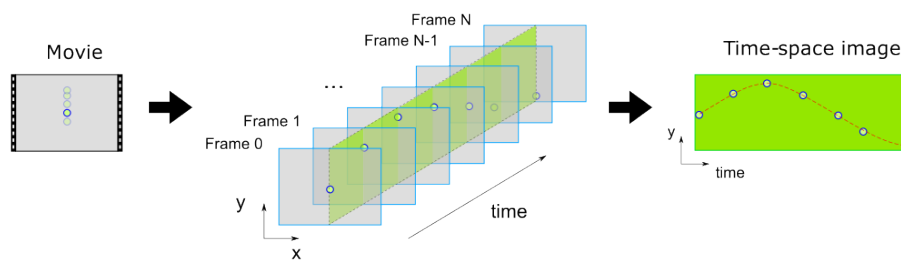


Fig. 5: Extracting a yt-image from a movie.

## Example Capillary rise

We want to study capillary rise and Washbourne's equation

*Analysis steps*

- Extract y-t slices
- Segment the front
- Locate the front as function of time

- Plot as function of  $\sqrt{t}$

For the interested: [Neutron imaging tutorial - capillary rise](#)

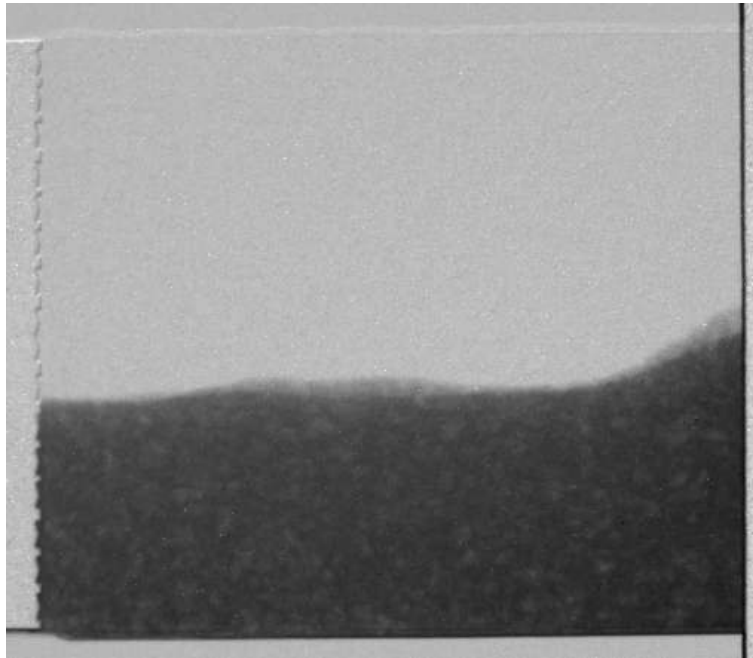


Fig. 6: A frame from the capillary rise movie.

### 0.4.4 Snow flake tracking

The previous example was very simplified. Real applications are far more complicated with more particles and crossing trajectories.

Let's try to track snow flakes falling from a dark sky (Video by [Niezlyziom](#))

Some initial qualitative observations:

- Direction: The snow flakes fall from the upper left corner to the lower right
- Sizes: There is great variation in size
- Velocity: Small flakes seem to fall slower than larger ones

### Data preparation

The first step - extract the frames from the movie

Here we use the `ffmpeg` application to extract the frames from the mp4-movie clip we want to study. You may have to use a different approach depending on your data source. Sometimes you already have the frames straight from the detector system.

It takes a while to extract the frames, therefore we check if it is already done.



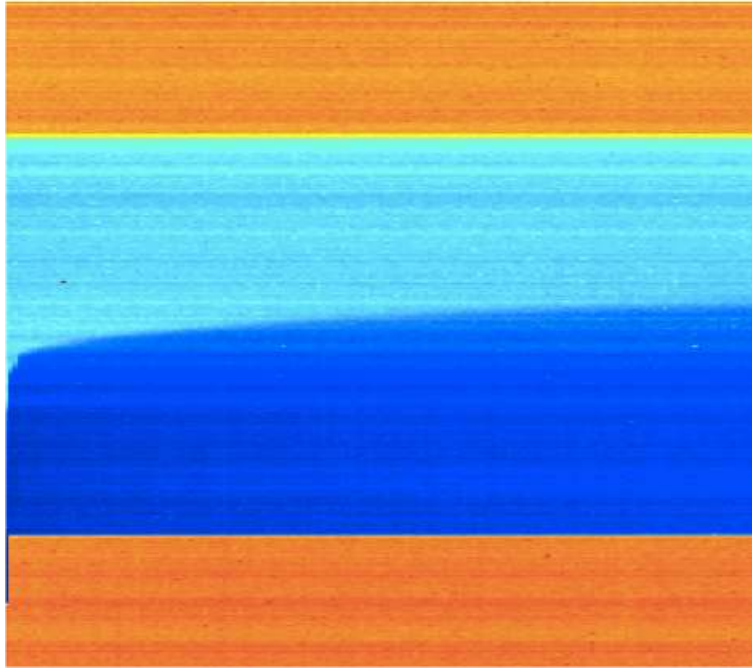


Fig. 7: An extracted yt-slice.

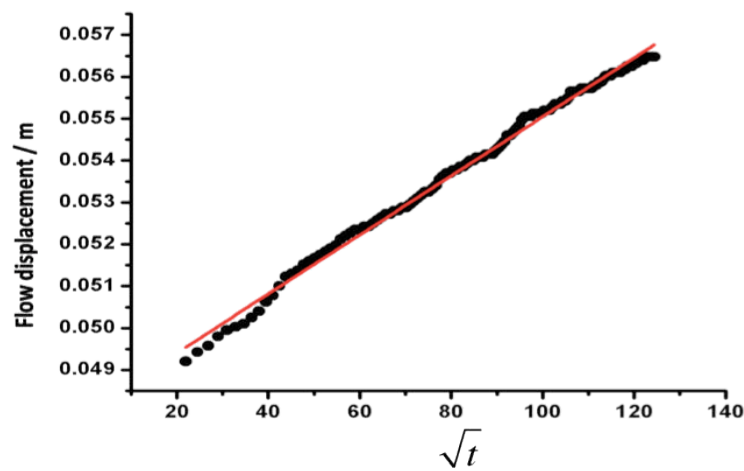
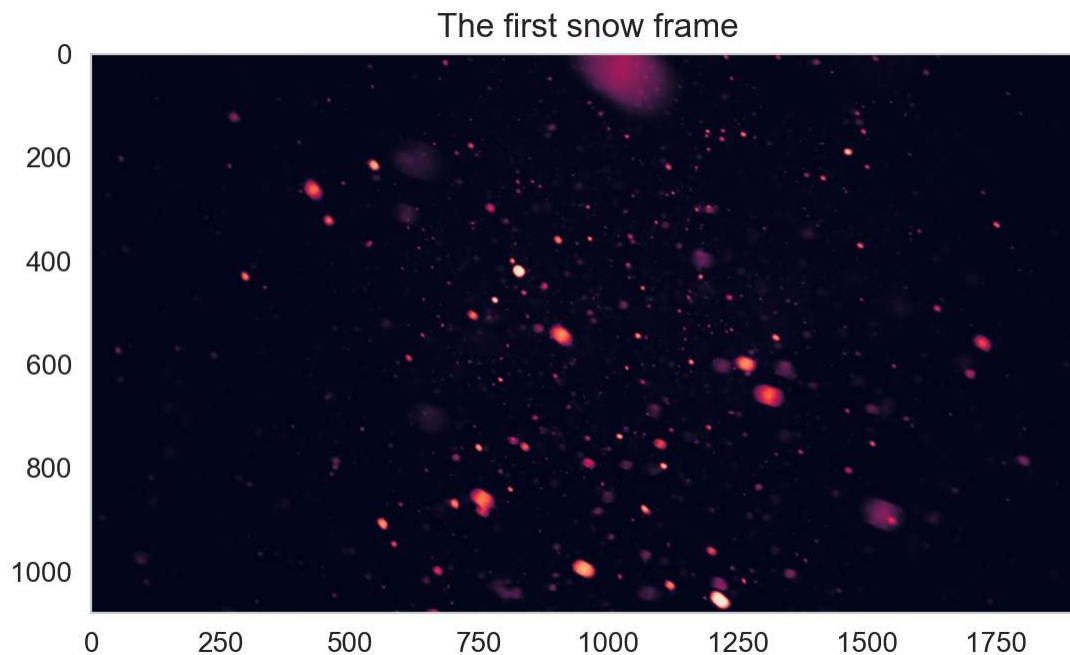


Fig. 8: Front positions follow Washbourne's equation.



Fig. 9: A frame from the snow fall movie.



### Load the frames

We now load the frames into a numpy array for the first analysis

Reading time series of data often requires a reader function that reads a series of 2D images with a given file mask like here we have files named `snow_0001.png`, `snow_0002.png`, `snow_0003.png`, ..., `snow_####.png`. The images are then inserted in a 3D numpy array which gives us the 3D image to analyse.

```
def read_series(fmask, first, last) :  
    tmp = plt.imread(fmask.format(first))  
  
    img=np.zeros([last-first+1,tmp.shape[0],tmp.shape[1]])  
    img[0]=tmp
```

(continues on next page)

(continued from previous page)

```

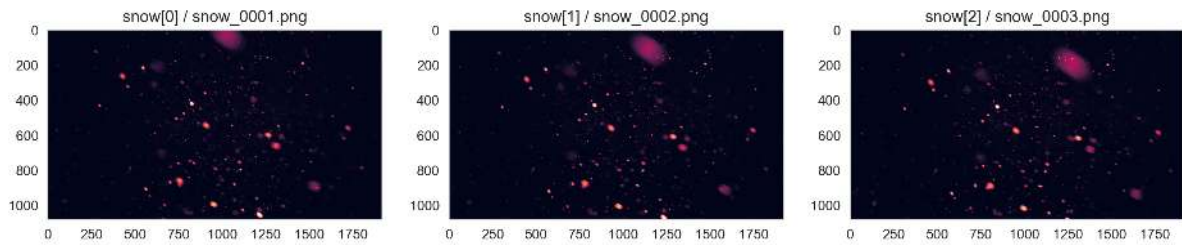
for idx in range(first+1,last+1) :
    img[idx-first]=plt.imread(fmask.format(idx))

return img.astype(float)

```

```
snow = read_series('movies/snow/snow_{0:04d}.png',first=1, last=300)
```

Here, we see the first three frames from the movie.



Observe the movie as a volume

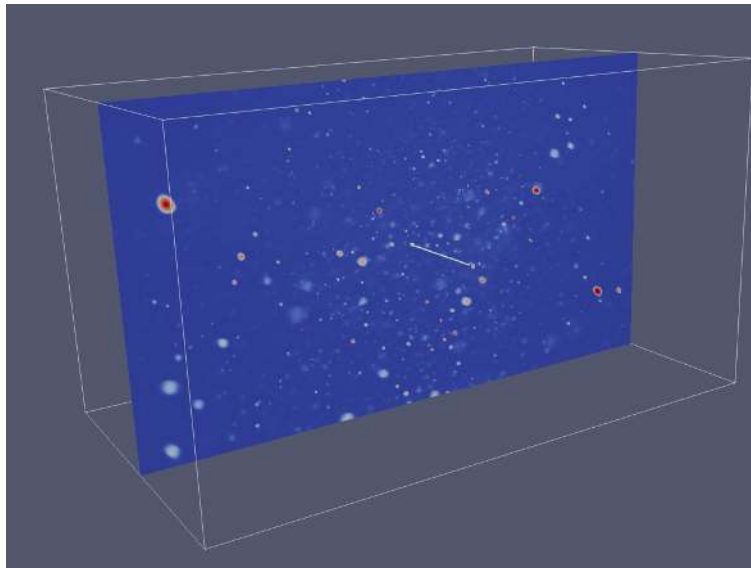
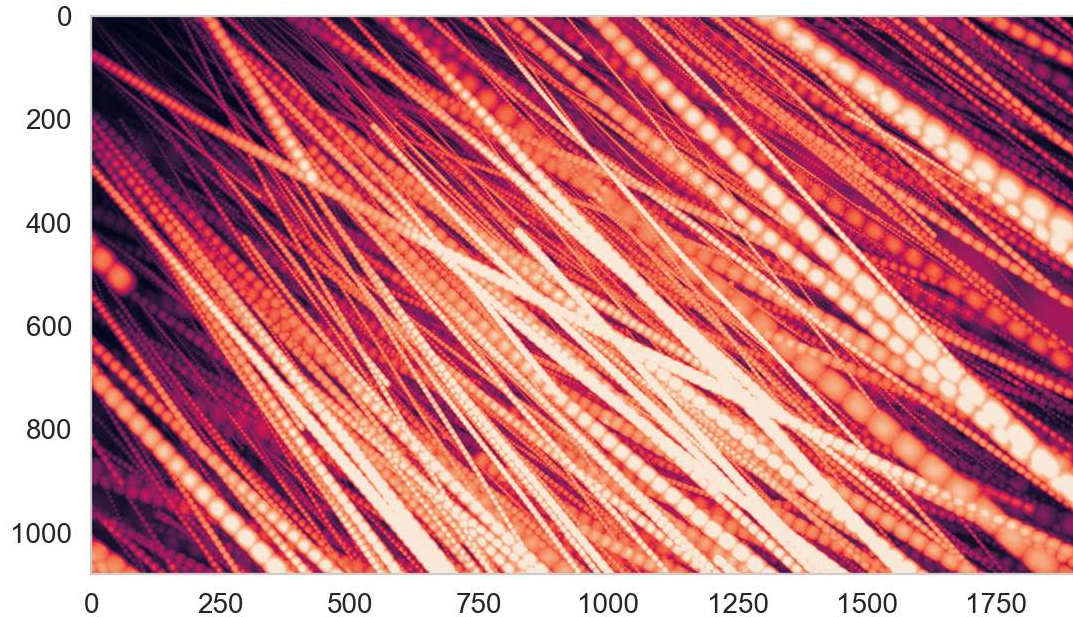


Fig. 10: A frame from the snow fall movie in the 3D frame.

### First steps using the 3D data

Let's try projecting the time-frames on the XY-plane

```
plt.imshow(snow.max(axis=0));
```



Some more observations:

- Each snow flake draws a track
- There is an intensity gradient along the track
- The diagonal pattern is confirmed
- The frame rate is on the lower end.

It is however hard to track individual flakes in this image

Depending on the purpose of the analysis, we might already be satisfied with the track pattern produced by the snow flakes. These give an average directions of the snow fall.

The fact that the frame rate is on the lower end can be seen in the dotted track lines. This means that the time between two frames allowed the snow flakes to move further away than their extent and that there is no clear overlap.

### A full 3D view of the snow fall

In 3D, we are able to better track the individual snow flakes.

Here, they appear as tubes in the 3D space, which allows to use volume segmentation approaches for the flake tracking. There is still the problem of crossing tracks which needs to be tackled.

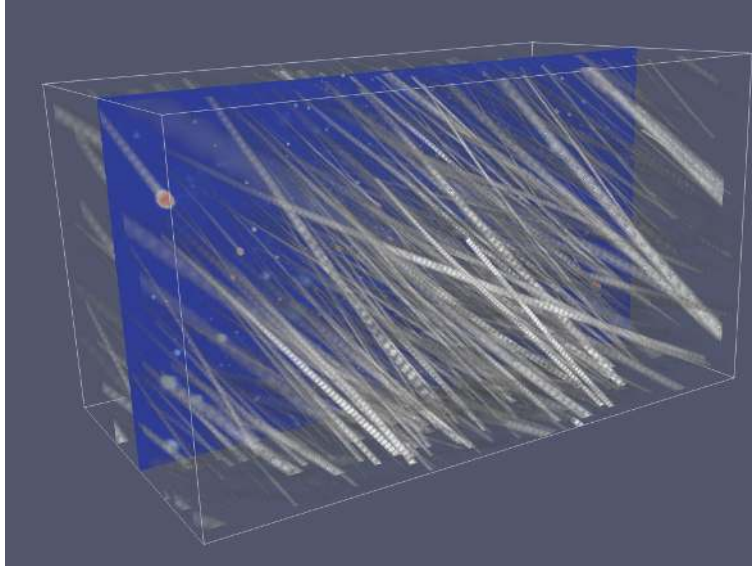


Fig. 11: A frame from the snow fall movie in the 3D frame with the track history added.

## 0.4.5 Tuning experiment by simulation

In many cases:

- experimental data or setup is inherited
- little can be done about the design,

**When there is still the opportunity to tune the experiment**

simulations provide a powerful tool for tuning and balancing a large number parameters

### Validation

Simulations also provide the ability to pair post-processing to the experiments and determine the limits of tracking.

## 0.5 Tracking objects and changes

### 0.5.1 What do we start with?

Going back to our original cell image

1. We have been able to *get rid of the noise* in the image and *find all the cells* (**lecture 2-4**)
2. We have analyzed the shape of the cells using the shape tensor (**lecture 5**)
3. We even *separated cells* joined together using Watershed (**lecture 6**)
4. We have created even more *metrics characterizing the distribution* (**lecture 7**)

We have at least a few samples (or different regions), large number of metrics and an almost as large number of parameters to *tune*

How do we do something meaningful with it?

### 0.5.2 Pixel/Voxel-based Methods

- Cross Correlation
- DIC
- DIC + Physics
- Affine Transforms
- Non-rigid transform

### Keypoint Detection

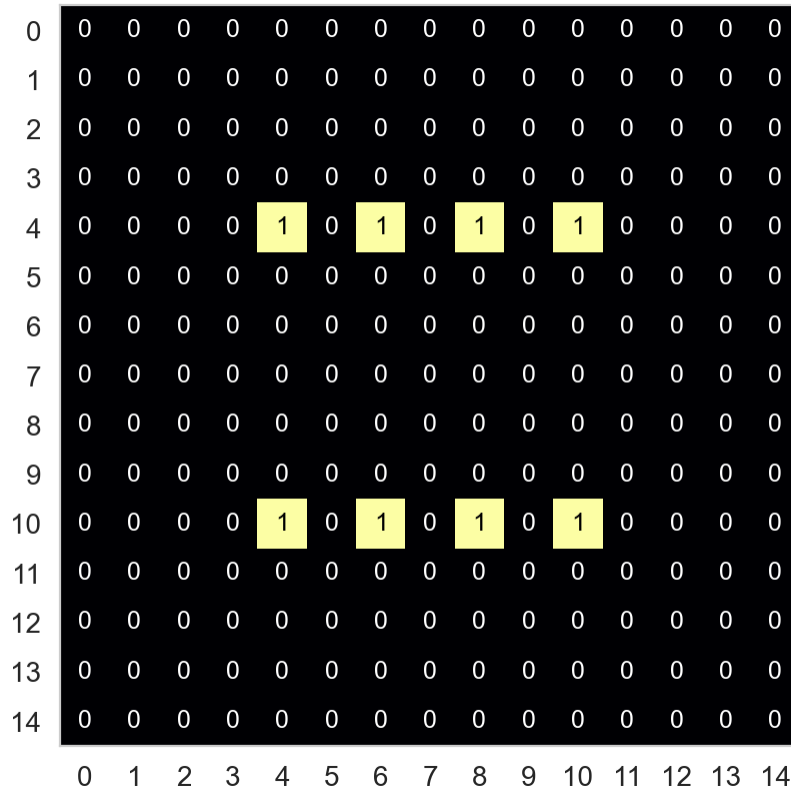
- Corner Detectors
- SIFT/SURF
- Tracking from Keypoints

### 0.5.3 General Problems

- Thickness - Lung Tissue
- Curvature - Metal Systems
- Two Point Correlation - Volcanic Rock

### 0.5.4 A basic simulation

We start with an initial image with a number of points on a plane



### 0.5.5 Create a series of moving “cells”

The cells will move with  $dx=-1$  and  $dy=-1$

### 0.5.6 Analysis

The analysis of the series requires the following steps:

1. Threshold
2. Component Label
3. Shape Analysis (label, position, area, and frame id)
4. Distribution Analysis

... and to put all in a data frame

### The analysis code

```
all_objs = []
for frame_idx, c_img in enumerate(img_list):           # For each time frame
    lab_img = label(c_img > 0)                         # Label the items
    for c_obj in regionprops(lab_img):                 # Put region properties for each
        ↪object of the time frame
            all_objs += [dict(label      = int(c_obj.label),
                               y         = c_obj.centroid[0],
                               x         = c_obj.centroid[1],
                               area      = c_obj.area,
                               frame_idx = frame_idx)]

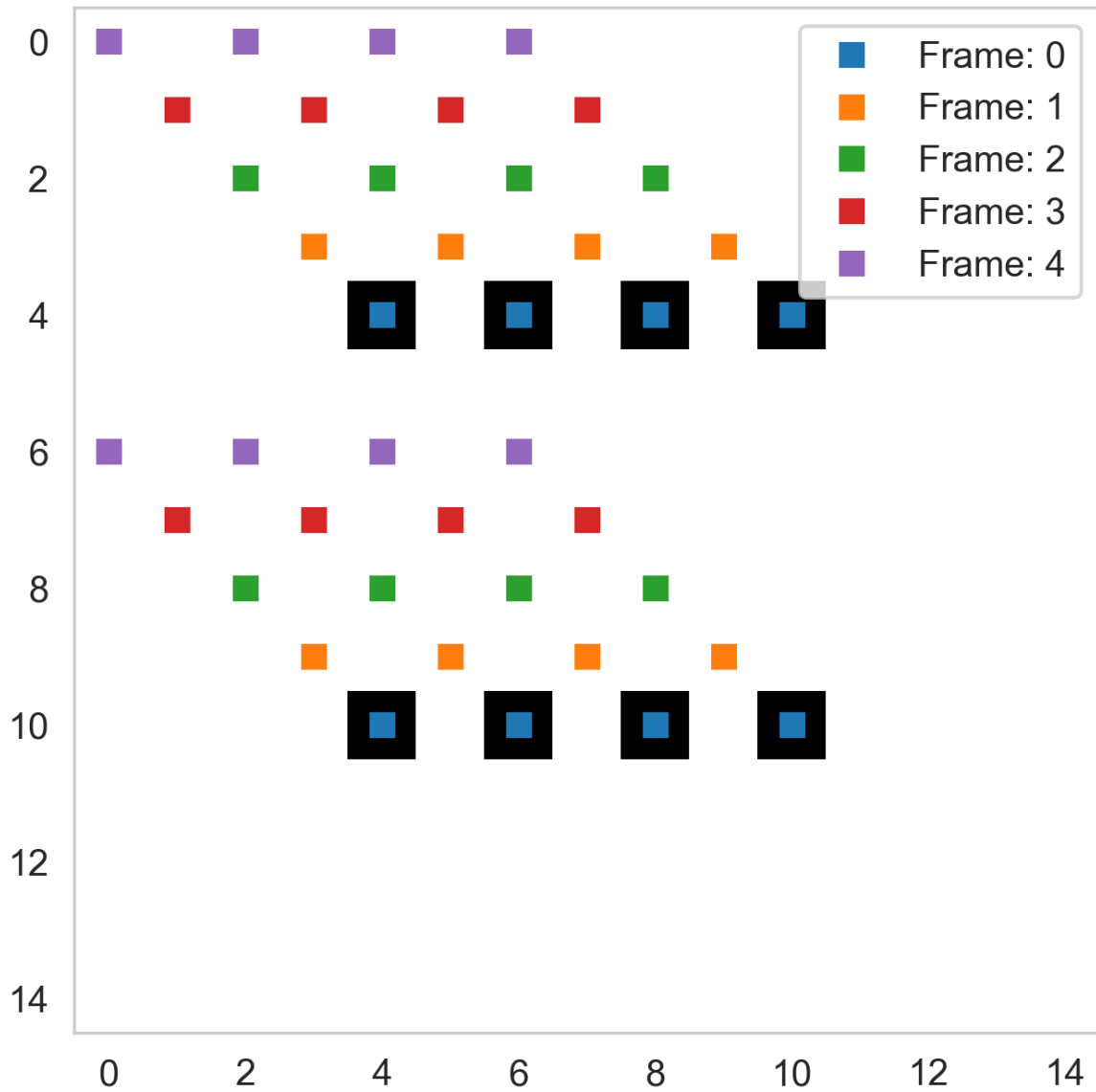
all_obj_df = pd.DataFrame(all_objs)                   # Create a Pandas data frame with
        ↪all the properties
all_obj_df.head(5)                                   # Look at the first five rows of
        ↪the data frame
```

|   | label | y    | x    | area | frame_idx |
|---|-------|------|------|------|-----------|
| 0 | 1     | 4.0  | 4.0  | 1.0  | 0         |
| 1 | 2     | 4.0  | 6.0  | 1.0  | 0         |
| 2 | 3     | 4.0  | 8.0  | 1.0  | 0         |
| 3 | 4     | 4.0  | 10.0 | 1.0  | 0         |
| 4 | 5     | 10.0 | 4.0  | 1.0  | 0         |

### Looking at the positions of the items in all frames

Here, we look at the positions of the items found in each time frame. The information comes from the data frame we just created. We only use position and frame index here.

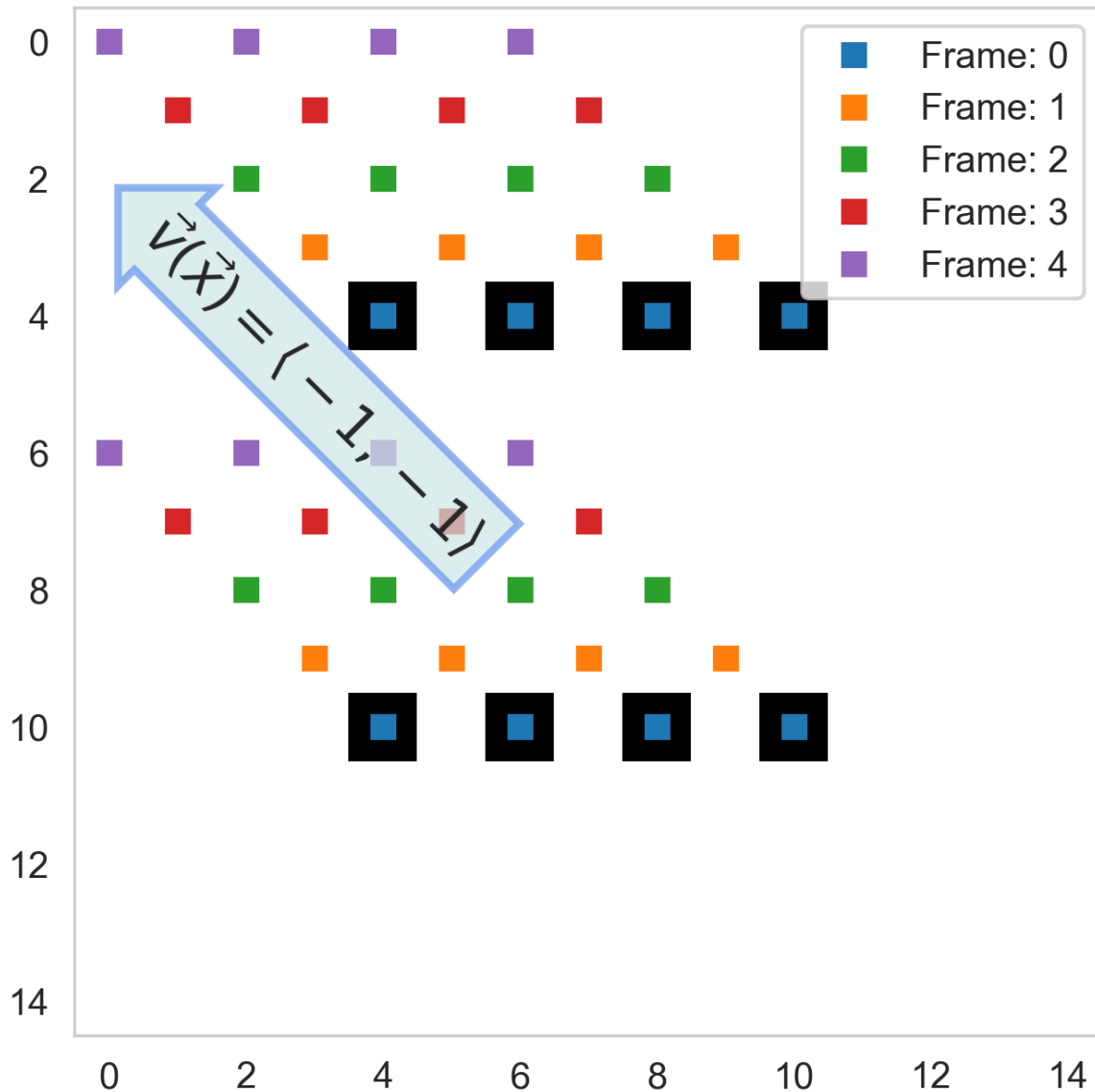




### Describing Motion

We can describe the motion in the above example with a simple vector

$$\vec{v}(\vec{x}) = \langle -1, -1 \rangle$$



This is a very simple case of motion as it only follows a straight line towards the upper left corner. Motion in real experiments are much more complicated and it can also be ambiguous which item is which when the trajectories are crossing.

### 0.5.7 Tracking methods

While there exist a number of different methods and complicated approaches for tracking.

For experimental design it is best to start with the (Occam's razor)

- simplest
- easiest understood method.

The limits of this can be found and components added as needed until it is possible to realize the experiment

*If a dataset can only be analyzed with a multiple-hypothesis testing neural network model then it might not be so reliable*

## Scoring Tracking

In the following examples we will use simple metrics for scoring fits where the objects are matched and the number of misses is counted.

There are a number of more sensitive scoring metrics which can be used, by finding the best submatch for a given particle since the number of matches and particles does not always correspond.

See the papers at the beginning for more information

### 0.5.8 Tracking using Nearest Neighbor

We then return to *nearest neighbor* which means we track

- a point ( $\vec{P}_0$ ) from an image ( $I_0$ ) at  $t_0$
- to a point ( $\vec{P}_1$ ) in image ( $I_1$ ) at  $t_1$

by

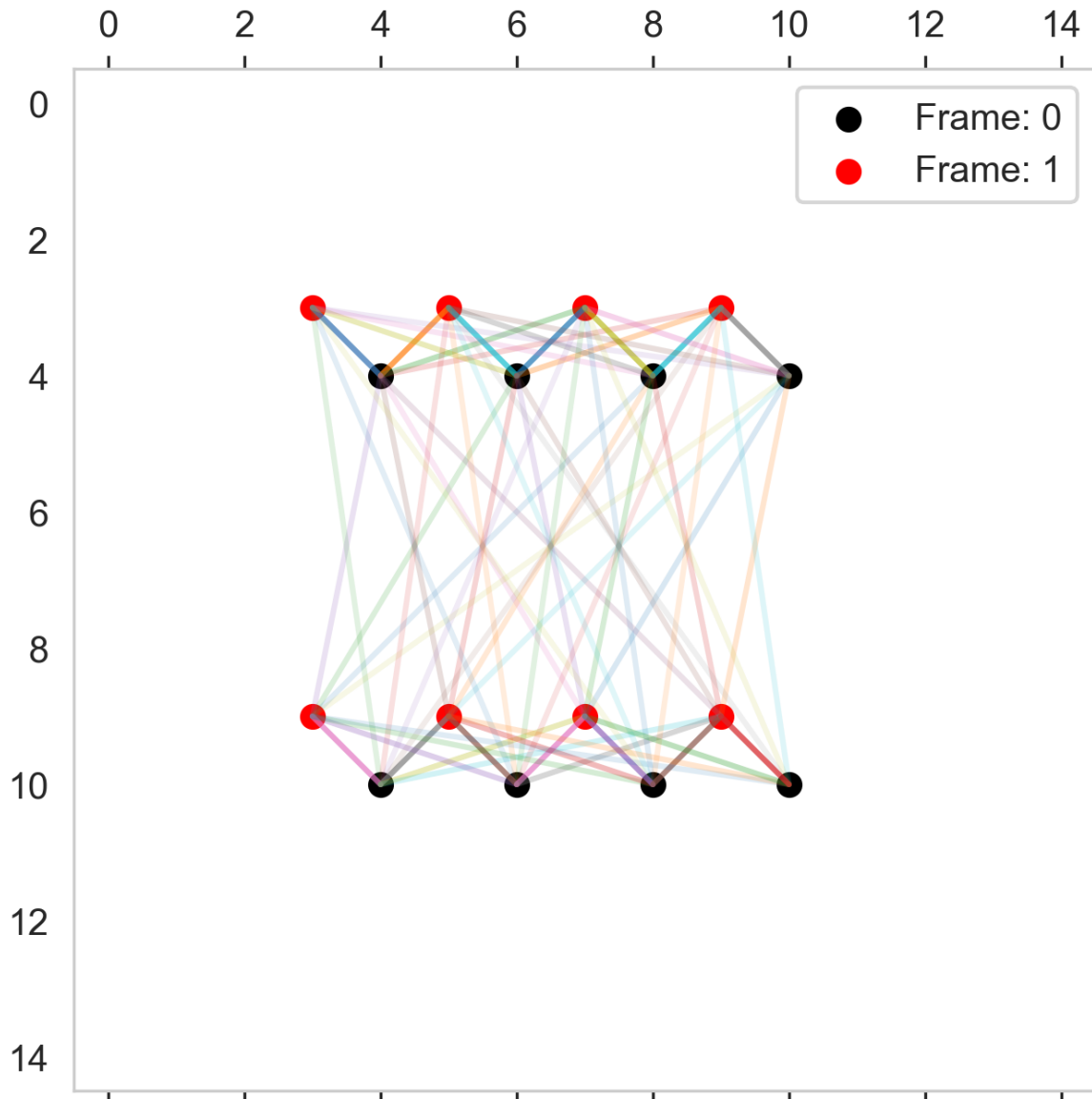
$$\vec{P}_1 = \operatorname{argmin}(\|\vec{P}_0 - \vec{y}\| \mid \forall \vec{y} \in I_1)$$

### 0.5.9 Distances between objects

The plot below shows the distances from each object to all other objects. Without additional information like

- we are looking for the nearest neighbor
- or the expected displacement is bounded within some limits.
- shape characteristics of the items

It is impossible to tell which is the matching object between frame 0 and 1.



### 0.5.10 Put the distances in a data frame

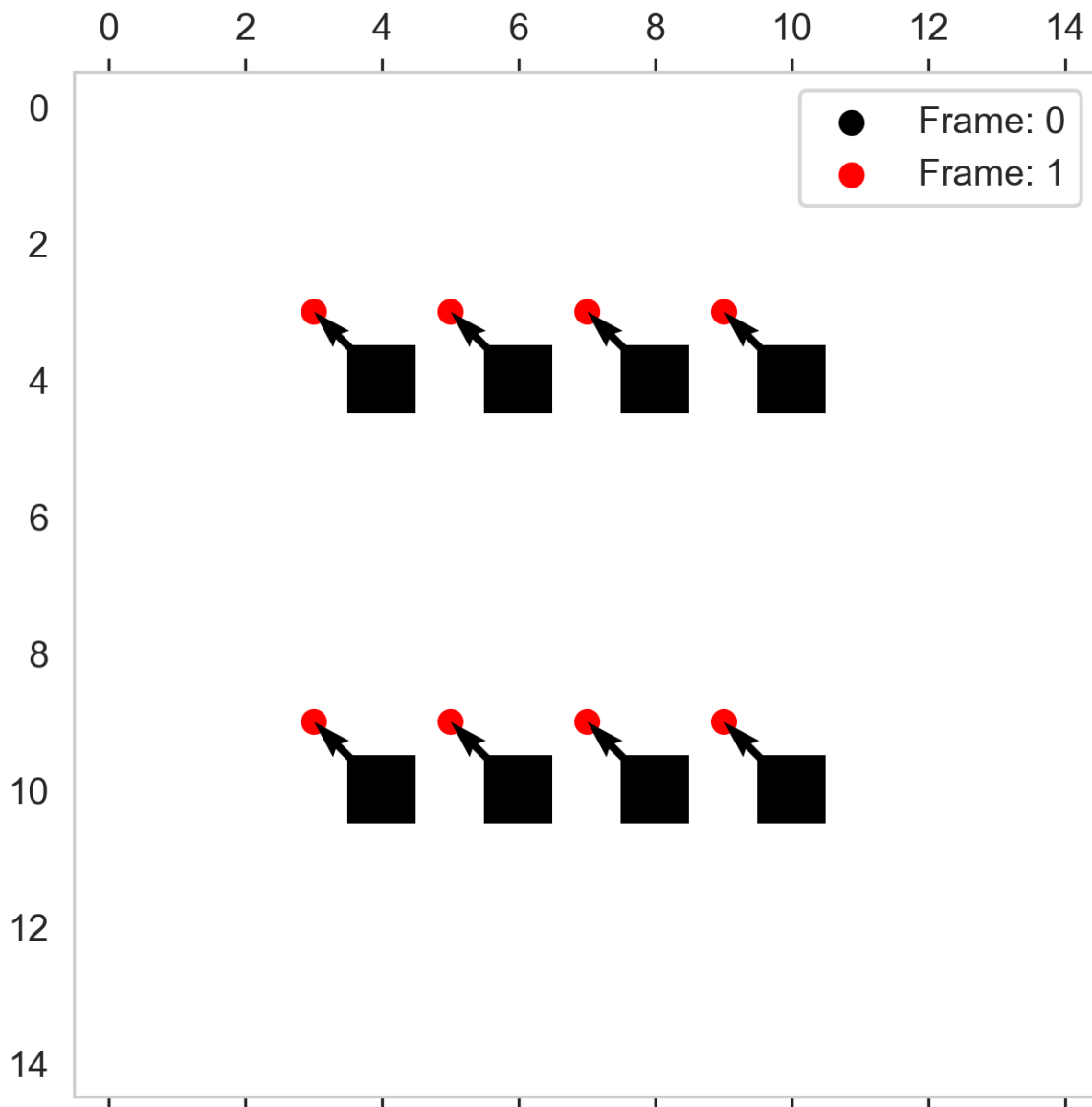
```
dist_df = pd.DataFrame(dist_df_list)
dist_df.head(5)
```

|   | x0  | y0  | lab0 | x1  | y1  | lab1 | dist     |
|---|-----|-----|------|-----|-----|------|----------|
| 0 | 4.0 | 4.0 | 1    | 3.0 | 3.0 | 1    | 1.414214 |
| 1 | 4.0 | 4.0 | 1    | 5.0 | 3.0 | 2    | 1.414214 |
| 2 | 4.0 | 4.0 | 1    | 7.0 | 3.0 | 3    | 3.162278 |
| 3 | 4.0 | 4.0 | 1    | 9.0 | 3.0 | 4    | 5.099020 |
| 4 | 4.0 | 4.0 | 1    | 3.0 | 9.0 | 5    | 5.099020 |

### Plotting the distances

With the nearest neighbor criterion we still have the problem because most items have two possible next position.

### Looking at the nearest points found



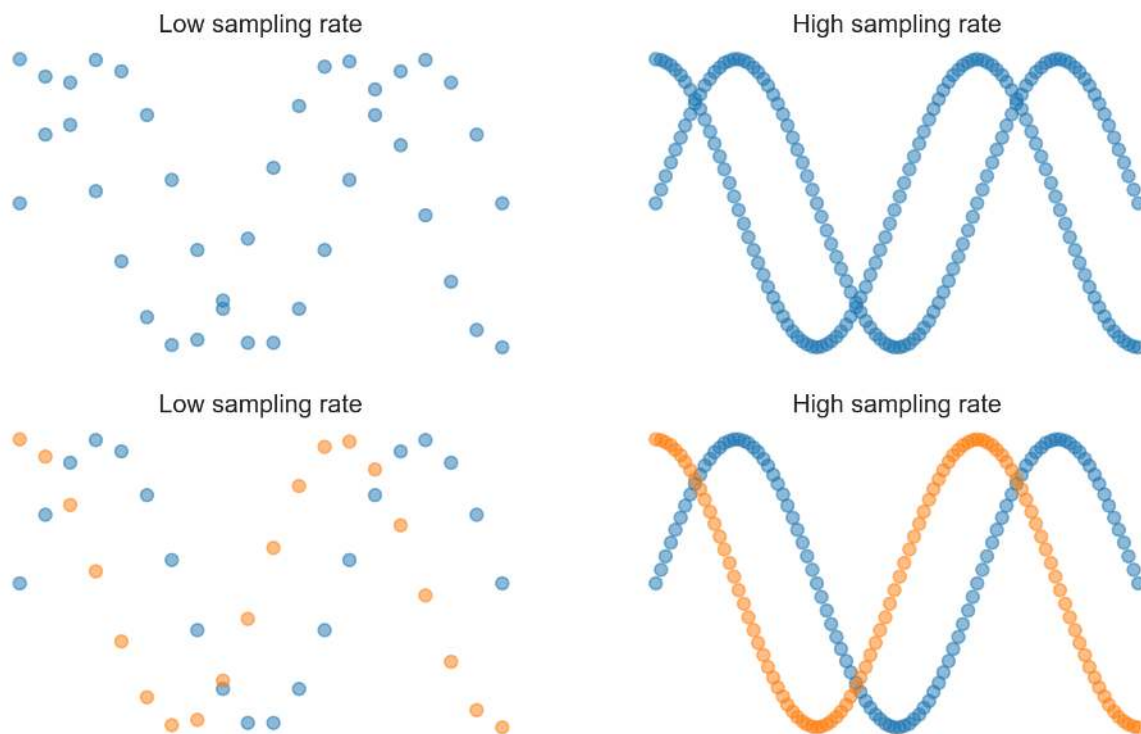
### Tracking in all frames

This tracking seems to be success full. The reason is that we were lucky that the direction is  $-1,-1$ . On the other hand, it is relatively rare that you have perfect positioning of the items and therefore this ambiguity doesn't happen very often.

### 0.5.11 Tracking at different sampling rates

Nearest neighbor tracking is sensitive to sampling rates

Here we have measured points for two items that follow a sine shaped trajectory. They are obtained with low and high sampling rates respectively.



We see here that the low sampling rate introduces quite long distances between two points which makes it hard to identify the next point. Higher sampling rate allows for more precise identification of the next point in the series.

Still, there are some ambiguities when the tracks cross. These points require special attention, which we come to later.

## 0.6 Key Points (or feature points)

Tracking and registration using the full data set is time demanding.

- We can detect feature points in an image and use them to make a registration.

The key points located at characteristic positions around the object and their positions relative to each other are invariant to translation and rotation. The points will still be at the same features when the object is skewed, but then they obtain new relative positions compared to the original.

### 0.6.1 Identifying key points

We first focus on the detection of points.

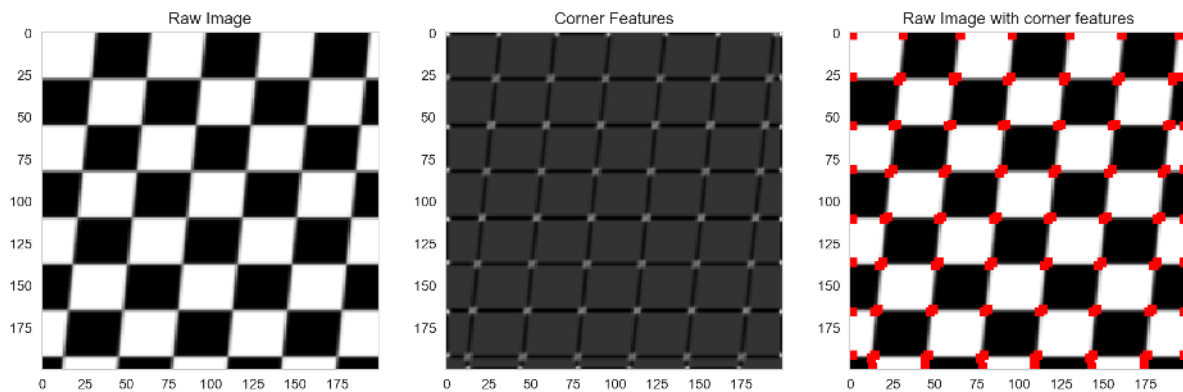
- Corners are of most interest.

Many methods have been proposed to detect corners. A Harris corner detector helps us here:

The Harris corner detector uses the structure tensor to identify corner points in the image. In the checkerboard example below we identify the corners using the Harris corner detector. In the corner feature image in the middle you see that the corners/crossings provide a strong response. The feature image is thresholded to provide the corner points, which are plotted in the panel to the right.

```
tform = AffineTransform(scale=(1.3, 1.1), rotation=0, shear=0.1, translation=(0, 0))
image = warp(data.checkerboard(), tform.inverse, output_shape=(200, 200))

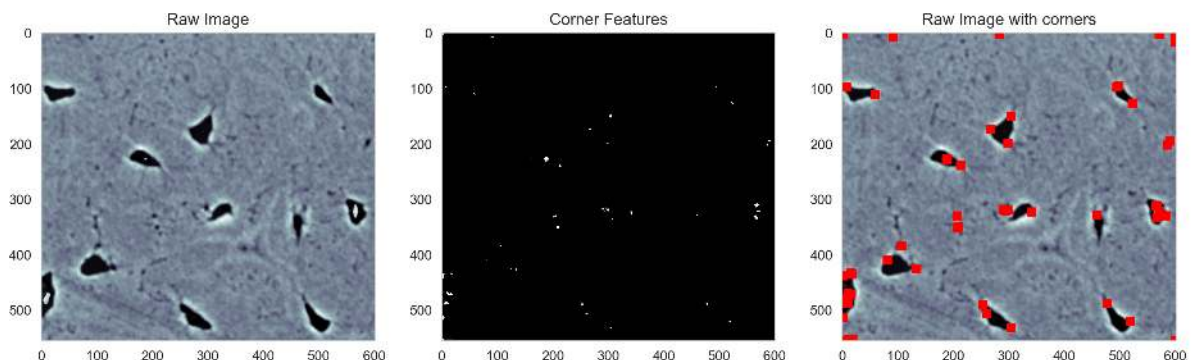
corners = corner_harris(image)
peak_coords = corner_peaks(corners, threshold_rel=0)
```



### 0.6.2 Let's try the corner detection on real data

We return to the bone image we have used many times before and test the Harris detector to identify key points in a natural image.

```
full_img = imread("figures/bonegfiltslice.png").mean(axis=2)
corners = corner_harris(full_img)
peak_coords = corner_peaks(corners, threshold_rel=0.002)
```



## 0.7 Tracking with Points

**Goal:** To reducing the tracking efforts

We can use the corner points to track features between multiple frames.

In this sample, we see that they are

- quite stable
- and fixed

on the features.

### 0.7.1 We need data - a series transformed images

In this example we apply different affine transformations like

- Translation in x and y
- Rotation
- Shear

to see how well the Harris detector copes with these.

The animation clearly shows that the corners are found.

It can however happen that there are various number of pixels allocated to each corner point. This can be pruned using morphological algorithms if needed.

## 0.8 Features and Descriptors

We can move beyond just key points to key points and feature vectors (called descriptors) at those points.

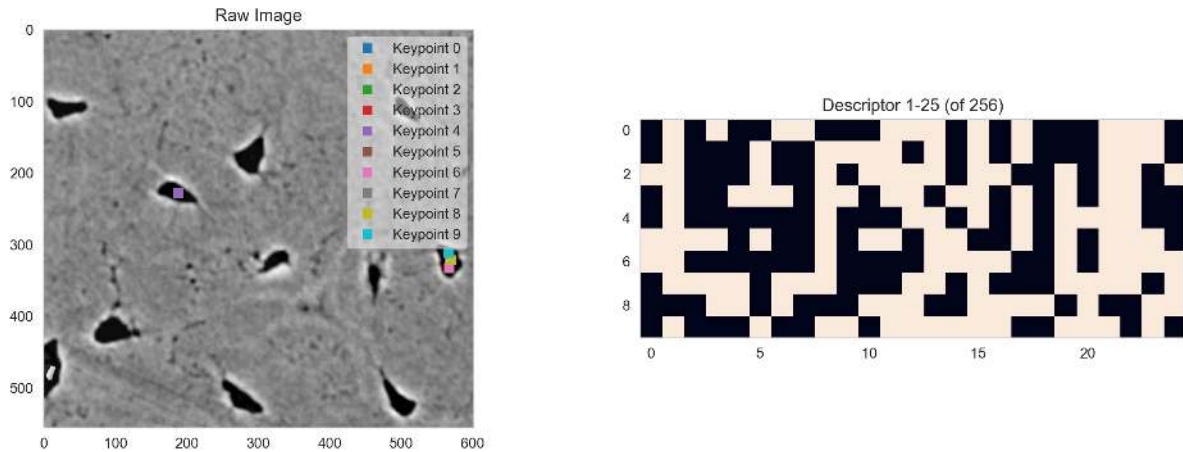
A descriptor is a vector that describes a given keypoint uniquely.

A common descriptor is computed using the **ORB** algorithm based on

- **FAST** keypoint detector (**F**eatures from **a**ccelerated **s**egment **t**est)
- **BRIEF** descriptor (**B**inary **R**obust **I**ndependent **E**lementary **F**eatures) doi

This will be demonstrated using two methods in the following notebook cells...





### 0.8.1 Defining a supporting function to show the matches

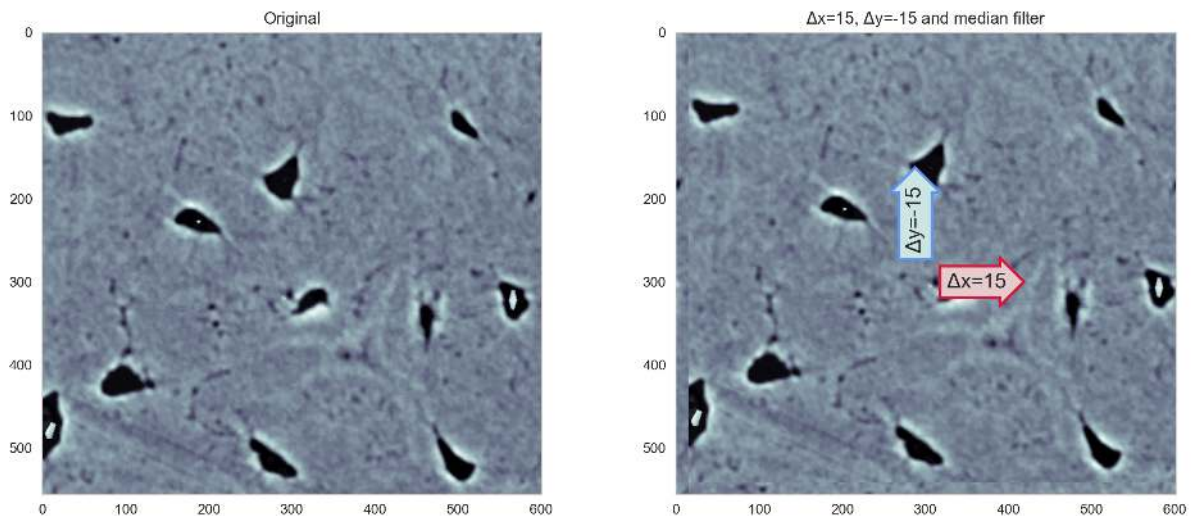
This is a support function used to display the matched feature points between two images.

### 0.8.2 Let's create some data

We make the following modifications to the bone image:

- affine transformation (linear displacement)
- blurring (median filter)

For this example we create a pair of images, where one translated and a smoothing median filter is applied. We will use this image pair to test different descriptors in the next sections.



### 0.8.3 Features found by the BRIEF descriptor

Scan the neighborhood of pixel  $x$ ,  $\mathcal{N}(x)$  to create a local fingerprint

$$\forall_{y \in \mathcal{N}(x)} \begin{cases} 1 & p(x) < p(y) \\ 0 & p(x) \geq p(y) \end{cases}$$

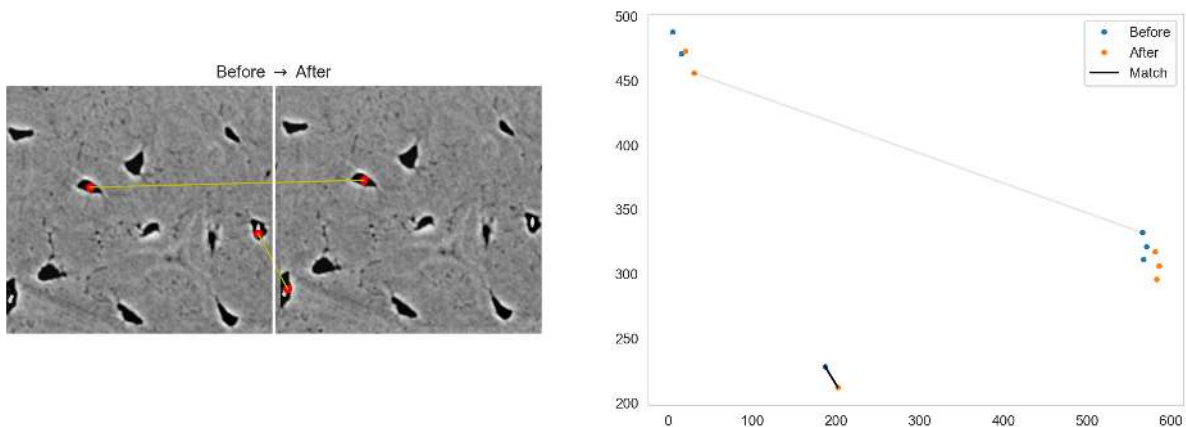
Produces a binary number where each bit is represented by a neighbor pixel.

In this first example we use a combination of the Harris corner detector and BRIEF to create descriptors.

```
from skimage.feature import corner_peaks, corner_harris, BRIEF

def calc_corners(*imgs):
    b = BRIEF()
    for c_img in imgs:
        corner_img = corner_harris(c_img)
        coords = corner_peaks(corner_img, min_distance=5, threshold_rel=0.1)
        b.extract(c_img, coords)
        yield {'keypoints': coords,
              'descriptors': b.descriptors}

feat1, feat2 = calc_corners(bw_img, shift_img)
show_matches(bw_img, shift_img, feat1, feat2)
```



You see two matches in the right panel. One short and the other obviously far away which is less likely to be realistic.

### 0.8.4 Features found by the ORB descriptor

The ORB descriptor which has a more robust algorithm is able to find many more good pairs between the two time steps.

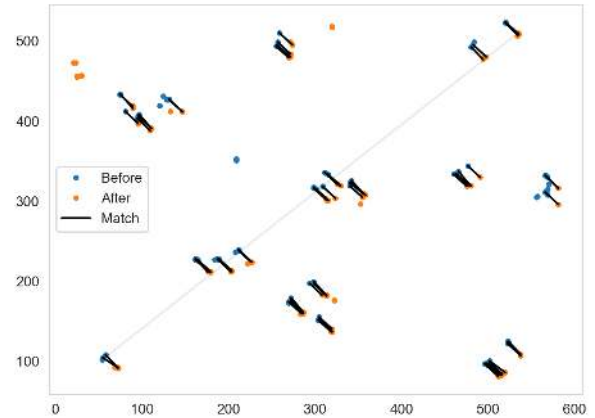
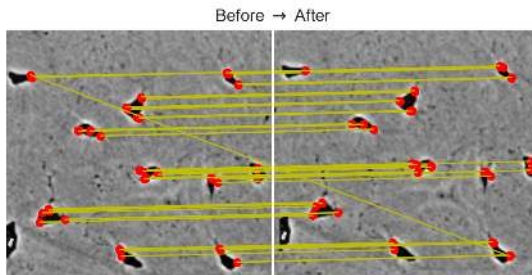
```
from skimage.feature import ORB, BRIEF, CENSURE

def calc_orb(*imgs):
    descriptor_extractor = ORB(n_keypoints=100)
    for c_img in imgs:
        descriptor_extractor.detect_and_extract(c_img)
        yield {'keypoints': descriptor_extractor.keypoints,
              'descriptors': descriptor_extractor.descriptors}
```

(continues on next page)

(continued from previous page)

```
feat1, feat2 = calc_orb(bw_img, shift_img)
show_matches(bw_img, shift_img, feat1, feat2)
```



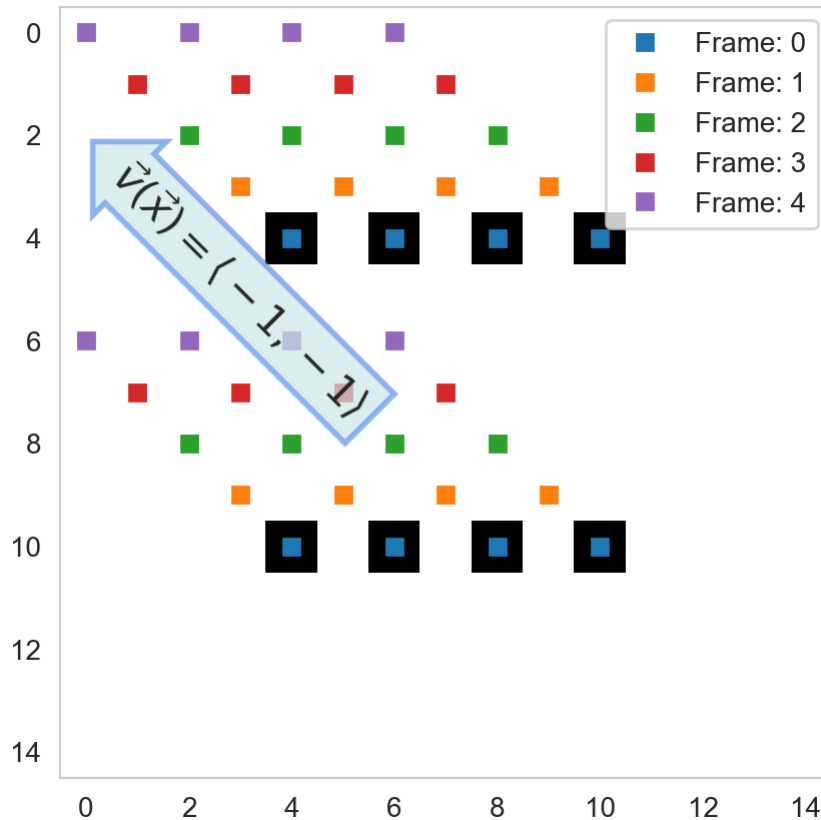
## 0.9 Computing Average Flow

From each of the time steps in a time series we can now proceed to compute the average flow.

We can perform this :

- spatially (averaging over regions),
- temporally (averaging over time),
- or spatial-temporally (averaging over regions for every time step)

Let's return to the  $\langle -1, -1 \rangle$  flow data set:



### 0.9.1 Compute the average velocity vector

Here we want the average of all displacements

```
average_field = []
for i in range(all_obj_df['frame_idx'].max()):
    frame_0 = all_obj_df[all_obj_df['frame_idx'].isin([i])]
    frame_1 = all_obj_df[all_obj_df['frame_idx'].isin([i+1])]
    dist_df_list = []

    # Compute distances to all points between two frames
    for _, row_0 in frame_0.iterrows():
        for _, row_1 in frame_1.iterrows():
            dist_df_list += [dict(x0=row_0['x'],
                                y0=row_0['y'],
                                lab0=int(row_0['label']),
                                x1=row_1['x'],
                                y1=row_1['y'],
                                lab1=int(row_1['label']),
                                dist=np.sqrt(
                                    np.square(row_0['x']-row_1['x']) +
                                    np.square(row_0['y']-row_1['y'])))]

    dist_df = pd.DataFrame(dist_df_list)
    # Get shortest distance for each label
    for _, c_rows in dist_df.groupby('lab0'):
        _, best_row = next(c_rows.sort_values('dist').iterrows())
```

(continues on next page)

(continued from previous page)

```

average_field += [dict(frame_idx=i,
                       x=best_row['x0'],
                       y=best_row['y0'],
                       dx=best_row['x1']-best_row['x0'],
                       dy=best_row['y1']-best_row['y0'])]
average_field_df = pd.DataFrame(average_field)
print('Average Flow:\n{}'.format(average_field_df[['dx', 'dy']].mean()))

average_field_df.sample(5)

```

```

Average Flow:
dx    -1.0
dy    -1.0
dtype: float64

```

|    | frame_idx | x   | y   | dx   | dy   |
|----|-----------|-----|-----|------|------|
| 15 | 1         | 9.0 | 9.0 | -1.0 | -1.0 |
| 12 | 1         | 3.0 | 9.0 | -1.0 | -1.0 |
| 20 | 2         | 2.0 | 8.0 | -1.0 | -1.0 |
| 22 | 2         | 6.0 | 8.0 | -1.0 | -1.0 |
| 28 | 3         | 1.0 | 7.0 | -1.0 | -1.0 |

Here, create an average field using the  $dx=-1$ ,  $dy=-1$  moving cells in the `all_obj_df`. The result is put in a new data frame.

The loop compares items in the current frame with items in the next frame and computes the displacements between all items. A second loop sorts the displacements to find the best combinations.

At the end, the average flow is computed.

## 0.9.2 Spatially Averaging

To spatially average we first create a grid of values and then interpolate our results onto this grid

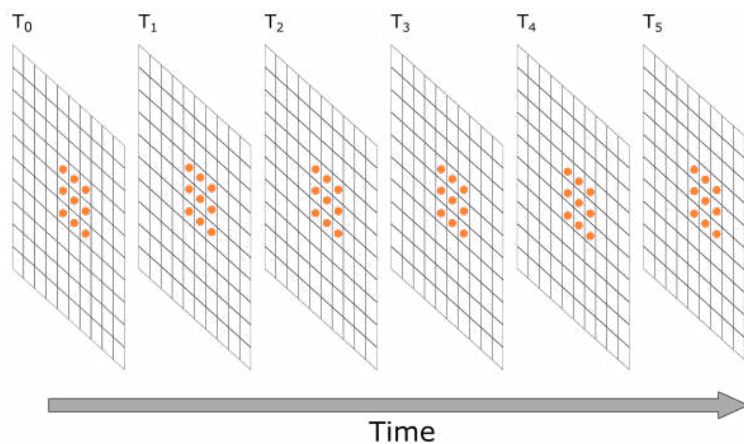


Fig. 1: Spatial averaging.

### 0.9.3 Averaging vector fields

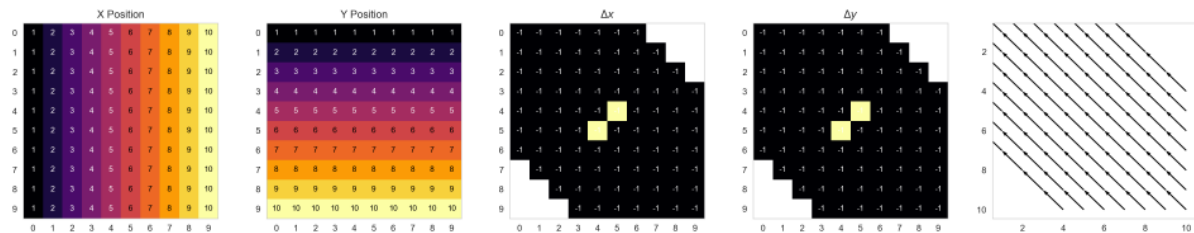
Vector fields can be very jittery in noisy studies. Spatial averaging can reduce the jitter.

```
from scipy.interpolate import RegularGridInterpolator, LinearNDInterpolator

def img_intp(f):
    def new_f(x, y):
        return np.stack([f(ix, iy) for ix, iy in zip(np.ravel(x), np.ravel(y))], 0).
        ↪ reshape(np.shape(x))
    return new_f

dx_func = img_intp(
    LinearNDInterpolator((average_field_df['x'], average_field_df['y']), average_
    ↪ field_df['dx']))
dy_func = img_intp(
    LinearNDInterpolator((average_field_df['x'], average_field_df['y']), average_
    ↪ field_df['dy']))
dx_func(8, 8), dy_func(8, 8)
```

```
(array(-1.), array(-1.))
```



### 0.9.4 Temporarily Averaging

Here we take the average at each time point

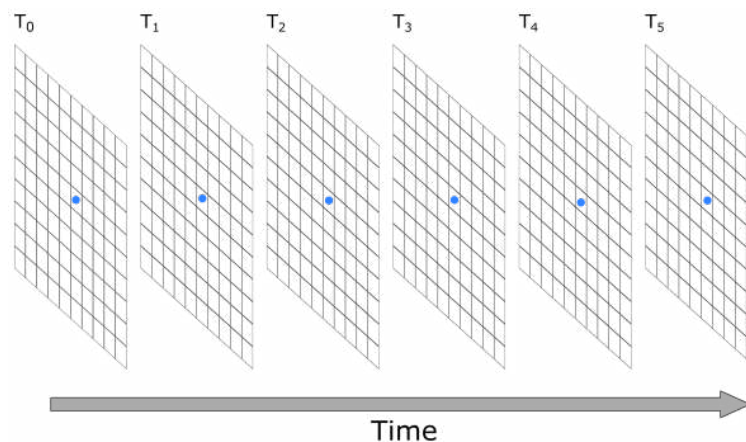


Fig. 2: Temporarily averaging.

## Temporarily averaging on the cell movement

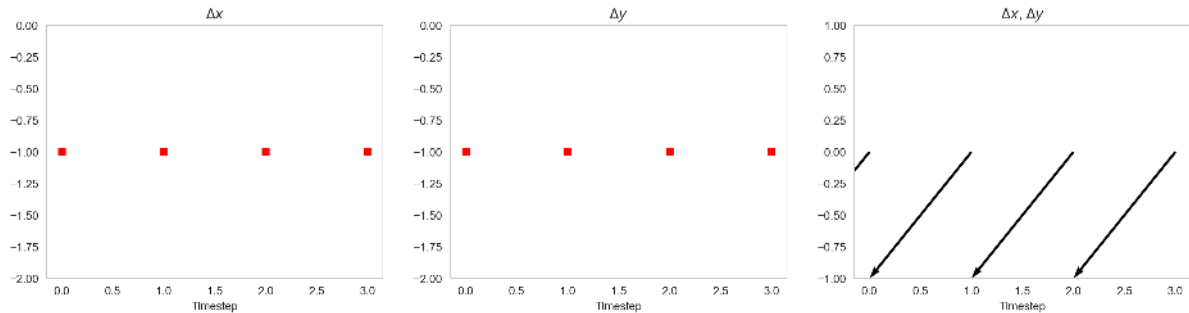
Compute the average displacement  $\langle \overline{\Delta x_i}, \overline{\Delta y_i} \rangle$  for each time frame  $i$ .

```
temp_avg_field = average_field_df[['frame_idx', 'dx', 'dy']].groupby(
    'frame_idx').agg('mean').reset_index()
temp_avg_field
```

| frame_idx | dx   | dy   |
|-----------|------|------|
| 0         | -1.0 | -1.0 |
| 1         | -1.0 | -1.0 |
| 2         | -1.0 | -1.0 |
| 3         | -1.0 | -1.0 |

The time frame average is computed using some pandas craft.

1. Extract the dx and dy from each frame
2. Group by the frame\_idx column
3. Compute the average of dx and dy for each frame.



## 0.9.5 Spatio-temporal Relationship

We can also divide the images into space and time steps

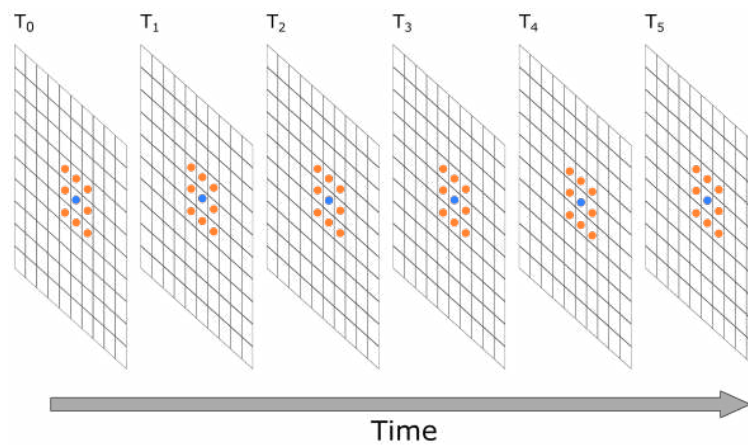


Fig. 3: Spatio-temporal averaging

## Spatio-temporal averaging on moving cells

Find all items for a time frame and compute spatial average field.

Here, we look at the down sampled vector field for each frame. The NaNs you see in  $\Delta x$  and  $\Delta y$  are points outside the interpolation support area.

## 0.9.6 Longer Series

We see that this approach becomes problematic when we want to work with longer series

```
import pandas as pd
from skimage.morphology import label
from skimage.measure import regionprops
from matplotlib.animation import FuncAnimation
from IPython.display import HTML

fig, c_ax = plt.subplots(1, 1, figsize=(5, 5), dpi=150)

s_img = disk_img.copy()
s_img = np.roll(np.roll(s_img, 4, axis=0), 4, axis=1)
img_list = [s_img]
for i in range(8):
    if i % 2 == 0:
        s_img = np.roll(s_img, -2, axis=0)
    else:
        s_img = np.roll(s_img, -1, axis=1)
    img_list += [s_img]

all_objs = []
for frame_idx, c_img in enumerate(img_list):
    lab_img = label(c_img > 0)
    for c_obj in regionprops(lab_img):
        all_objs += [dict(label=int(c_obj.label),
                           y=c_obj.centroid[0],
                           x=c_obj.centroid[1],
                           area=c_obj.area,
                           frame_idx=frame_idx)]
all_obj_df2 = pd.DataFrame(all_objs)
all_obj_df2.head(5)

def update_frame(i):
    plt.cla()
    ps.heatmap(img_list[i], bgbox=False, precision=0, ax=c_ax, fontsize=9)
    c_ax.set_title('Iteration #{}'.format(i+1))

# write animation frames
anim_code = FuncAnimation(fig,
                           update_frame,
                           frames=len(img_list),
                           interval=1000,
                           repeat_delay=2000)
```

(continues on next page)



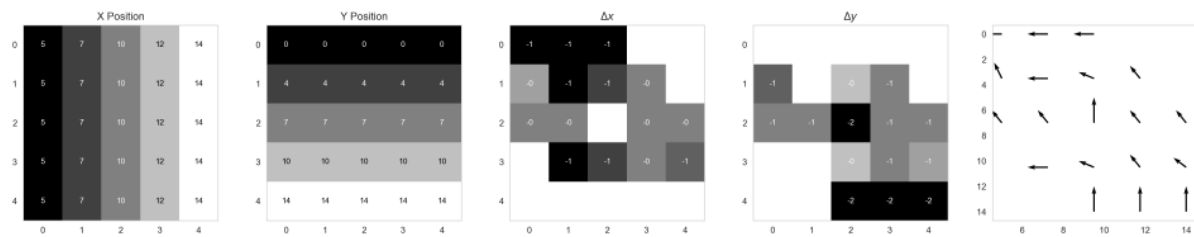
(continued from previous page)

```
anim_code.save('movies/longseries.mp4', fps=2)
plt.close('all')
```

## The resulting tracking

Tracking this longer series soon get complicated with the amount of direction changes in the scene.

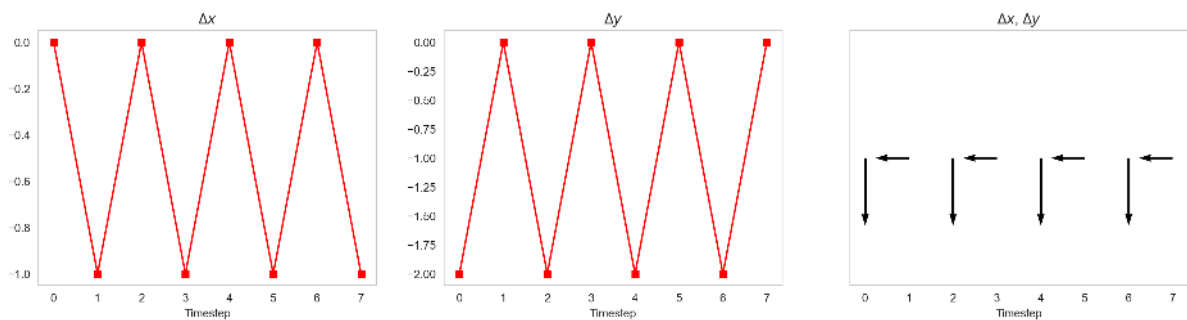
```
Average Flow:
dx    -0.5
dy    -1.0
dtype: float64
```



## Temporal averaging of the long series

```
temp_avg_field = average_field_df2[['frame_idx', 'dx', 'dy']].groupby(
    'frame_idx').agg('mean').reset_index()
```

| frame_idx | dx   | dy   |
|-----------|------|------|
| 0         | 0.0  | -2.0 |
| 1         | -1.0 | 0.0  |
| 2         | 0.0  | -2.0 |
| 3         | -1.0 | 0.0  |
| 4         | 0.0  | -2.0 |
| 5         | -1.0 | 0.0  |
| 6         | 0.0  | -2.0 |
| 7         | -1.0 | 0.0  |



### Spatio-temporal averaging of the longer series

#### Tracking using Trackpy

Someone else did the job for you... [trackpy](#)

[Tutorial](#)

## 0.10 Problems with tracking

### 0.10.1 Random Appearance / Disappearance

Under perfect imaging and experimental conditions objects should not appear and reappear but due to

- Noise
- Limited fields of view / depth of field
- Discrete segmentation approaches
- Motion artifacts
  - Blurred objects often have lower intensity values than still objects
- Crossed paths

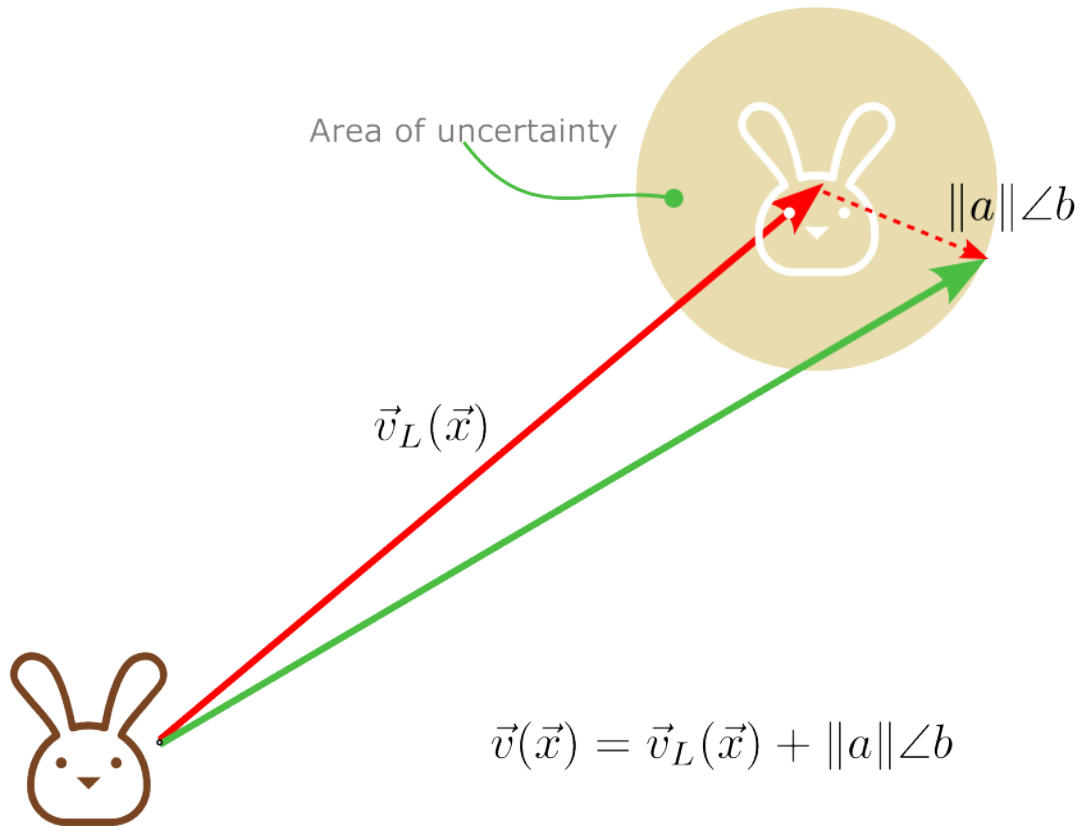
It is common for objects to appear and vanish regularly in an experiment.

### 0.10.2 Jitter / Motion Noise

Even perfect spherical objects do not move in a straight line:

- The jitter can be seen as a stochastic variable with a random
  - magnitude ( $a$ ) and
  - angle ( $b$ ).
- This is then sampled at every point in the field

$$\vec{v}(\vec{x}) = \underbrace{\vec{v}_L(\vec{x})}_{\text{deterministic}} + \underbrace{\|a\| \angle b}_{\text{random}}$$



There are several factors in the real experiment situation that can't be accounted for. The consequence is that the intended motion must be considered a random variable that has an expected velocity which is affected by the random effects. This random effect results in a region of uncertainty where the item is observed in the next time frame. Here, we model this as the random distance  $a$  in the direction  $b$ . I.e. the uncertainty region has a radius of  $E[a]$ .

### 0.10.3 Limitations of Tracking

We see that visually tracking samples can be difficult and there are a number of parameters which affect the ability for us to clearly see the tracking.

- flow rate
- flow type
- density
- appearance and disappearance rate
- jitter
- particle uniqueness

These experiment factors limit the accuracy and precision of the tracking. The sampling rate of the detector also plays a role here. In particular, it must match the flow rate and type of the process. The particle density and uniqueness makes it harder to identify the same particle between two frames.

### 0.11 How to improve tracking

We have to try to quantify the limits of these parameters for different tracking methods in order to design experiments better.

#### 0.11.1 Acquisition-based Parameters

- Acquisition rate
  - flow rate vs. sampling rate
  - jitter (per frame)
- Resolution
  - density,
  - appearance rate

#### 0.11.2 Experimental Parameters

- Experimental setup (pressure, etc) → flow rate/type
- Polydispersity → particle uniqueness
- Vibration/temperature → jitter
- Mixture → density contrast

#### 0.11.3 Basic Simulations

Input flow from simulation

$$\vec{v}(\vec{x}) = \langle 0, 0, 0.05 \rangle + ||0.01|| \nabla b$$

#### 0.11.4 Designing Experiments

How do simulations help us to design experiments?

- density can be changed by adjusting the concentration of the substances being examined or the field of view
- flow per frame (image velocity) can usually be adjusted by changing pressure or acquisition time
- jitter can be estimated from images

## How much is enough?

### Difficult to create one number for every experiment

- 5% error in bubble position →
  - <5% in flow field
  - >20% error in topology
- 5% error in shape or volume →
  - 5% in distribution or changes
  - > 5% in individual bubble changes
  - > 15% for single bubble strain tensor calculations

## 0.11.5 Extending Nearest Neighbor

We have two time frames  $I_0$  and  $I_1$

### Bijjective Requirement

We define  $\vec{P}_f$  as the result of performing the nearest neighbor tracking on  $\vec{P}_0$   $\vec{P}_f = \operatorname{argmin}(\|\vec{P}_0 - \vec{y}\| \mid \vec{y} \in I_1)$

We define  $\vec{P}_i$  as the result of performing the nearest neighbor tracking on  $\vec{P}_f$   $\vec{P}_i = \operatorname{argmin}(\|\vec{P}_f - \vec{y}\| \mid \vec{y} \in I_0)$

We say the tracking is bijective if these two points are the same  $\vec{P}_i \stackrel{?}{=} \vec{P}_0$

### Maximum Displacement

$$\vec{P}_1 = \begin{cases} \|\vec{P}_0 - \vec{y}\| < \text{MAXD}, & \operatorname{argmin}(\|\vec{P}_0 - \vec{y}\| \mid \vec{y} \in I_1) \\ \text{Otherwise,} & \emptyset \end{cases}$$

## 0.11.6 Extending Nearest Neighbor (Continued)

Models of movement behavior to support tracking

### Prior / Expected Movement

$$\vec{P}_1 = \operatorname{argmin}(\|\vec{P}_0 + \vec{v}_{offset} - \vec{y}\| \mid \vec{y} \in I_1)$$

### Adaptive Movement

Can then be calculated in an iterative fashion where the offset is the average from all of the  $\vec{P}_1 - \vec{P}_0$  vectors. It can also be performed

$$\vec{P}_1 = \operatorname{argmin}(\|\vec{P}_0 + \vec{v}_{offset} - \vec{y}\| \mid \forall \vec{y} \in I_1)$$

### More advanced models

- Use expected physical model
- Use track derivative to find  $v_{offset}$
- Kalman filters can be used for particle tracking

### 0.11.7 Beyond Nearest Neighbor

While nearest neighbor

- provides a useful starting tool
- it is not sufficient for truly complicated flows and datasets.

### Better Approaches

#### Multiple Hypothesis Testing

- Nearest neighbor just compares the points between two frames and there is much more information available in most time-resolved datasets.
- This approach allows for multiple possible paths to be explored at the same time and the best chosen only after all frames have been examined

### Shortcomings

#### Merging and Splitting Particles

- The simplicity of the nearest neighbor model does really allow for particles to merge and split (relaxing the bijective requirement allows such behavior, but the method is still not suited for such tracking).
- For such systems a more specific, physically-based is required to encapsulate this behavior.

### 0.11.8 Voxel-based Approaches

For voxel-based approaches the most common analyses are digital image correlation (or for 3D images digital volume correlation), where the correlation is calculated between two images or volumes.

## Standard Image Correlation

Given images  $I_0(\vec{x})$  and  $I_1(\vec{x})$  at time  $t_0$  and  $t_1$  respectively. The correlation between these two images can be calculated for each  $\vec{r}$

$$C_{I_0, I_1}(\vec{r}) = \langle I_0(\vec{x}) I_1(\vec{x} + \vec{r}) \rangle$$

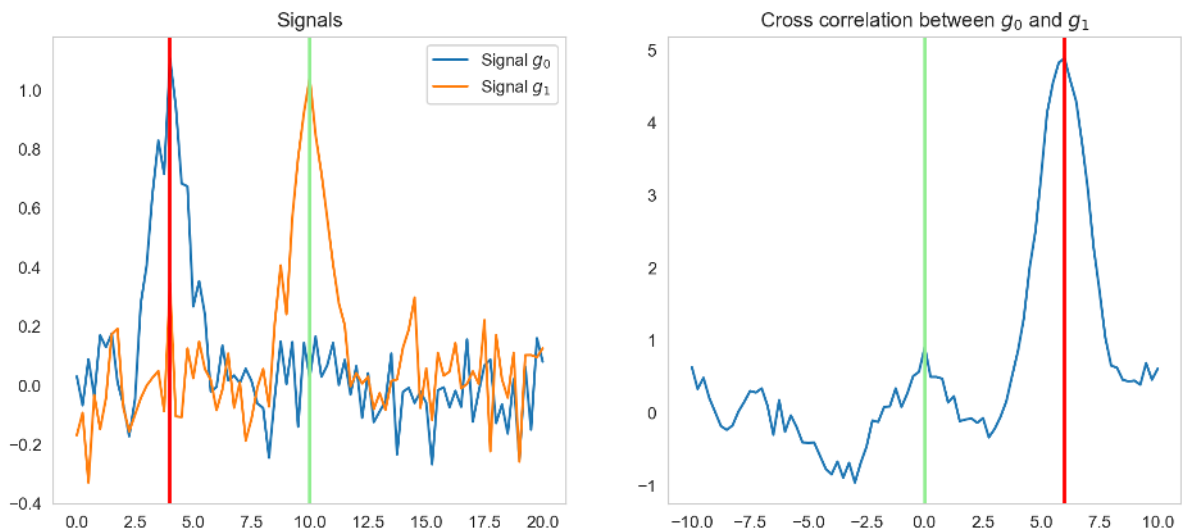
This can also be done in the Fourier space

$$C_{I_0, I_1}(\vec{r}) = \mathcal{F}^{-1}\{\mathcal{F}\{I_0\} \cdot \mathcal{F}\{I_1\}^*\}$$

## Correlation in 1D

```
N=81
x=np.linspace(0,20,N)
g0=np.exp(-(x-4)**2)+np.random.normal(0,0.1,size=N)
g1=np.exp(-(x-10)**2)+np.random.normal(0,0.1,size=N)

fg0=np.fft.fft(g0)
fg1=np.fft.fft(g1)
fg0g1=fg1*np.conj(fg0)
cg0g1=np.real(np.fft.ifft(fg0g1))
```

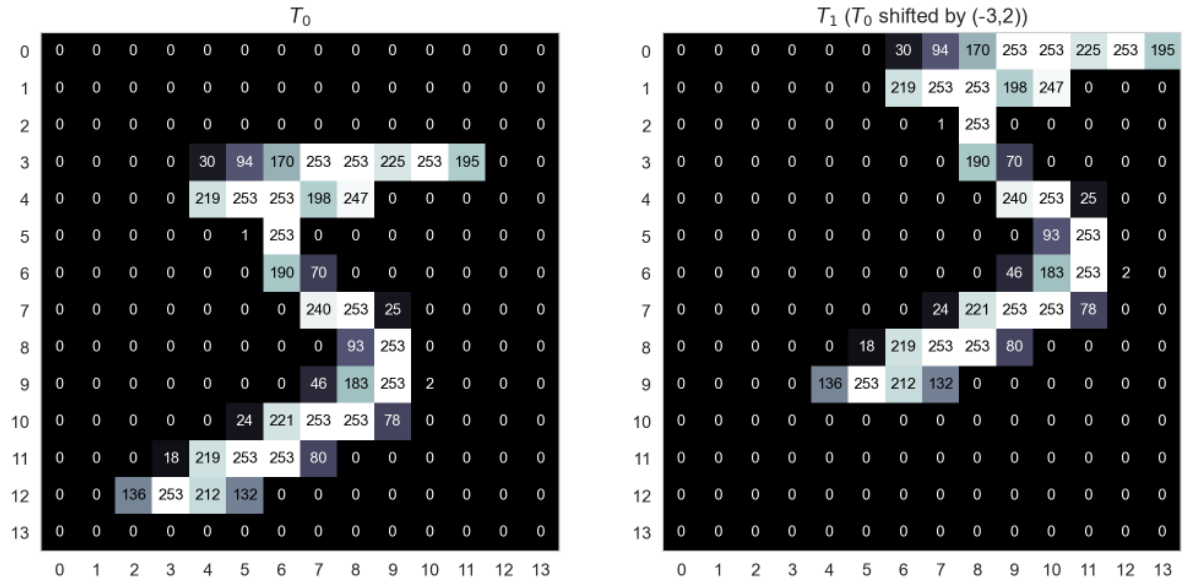


## Let's make some test data

We use a 'five' from the MNIST data set of handwritten numbers

```
bw_img = np.load('data/five.npy') # A 'five' from the mnist data set

shift_img = np.roll(np.roll(bw_img, -3, axis=0), 2, axis=1) # Shifting image by (dx,
↳dy)=(2,-3) using rolling
```



### Principle of correlation in space

- We have images  $f$  and  $g$ .
- Image  $g$  is translated by some  $\Delta x_0$  and  $\Delta y_0$

We iterate over displacements in x and y

- Translate  $g$  by  $\Delta x$  and  $\Delta y \rightarrow g_{trans}$
- Compute the pixel-wise product of  $f$  and  $g_{trans}$
- Compute the sum
- Store the value in the correlation map at position  $\Delta x$  and  $\Delta y$

### Practical issues

The translation here is solved with a pixel-wise rolling operation.

We need zero padding to avoid strange correlation results

```
def pad_image(img):
    dims = np.array(img.shape)
    padded_img = np.zeros(dims*3)

    padded_img[dims[0]:(2*dims[0]),dims[1]:(2*dims[1])]=img

    return padded_img

fig,ax = plt.subplots(1,4,figsize=(15,4))

# sns.heatmap(bw_img, ax=ax1, cbar=False, annot=True, fmt='d', cmap='bone')
ax[0].imshow(shift_img,cmap='bone')
ax[0].set_title('$T_1$')
# sns.heatmap(shift_img, ax=ax2, cbar=False, annot=True, fmt='d', cmap='bone') ,
shift_img_roll = np.roll(np.roll(shift_img, -4, axis=0), 4, axis=1)
```

(continues on next page)



(continued from previous page)

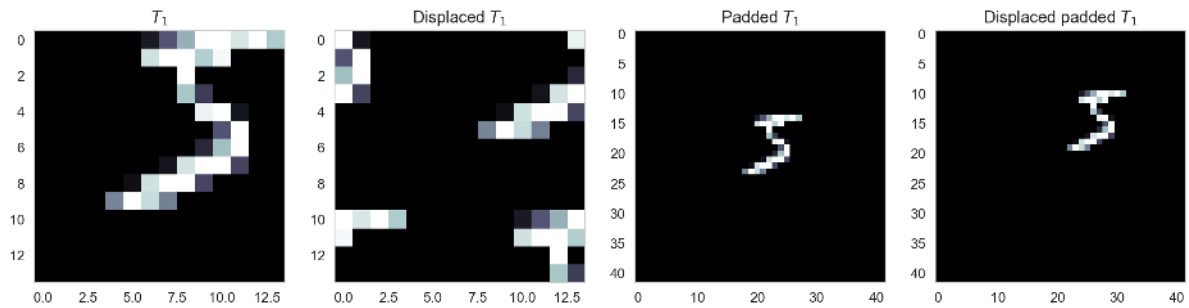
```

ax[1].imshow(shift_img_roll, cmap='bone')
ax[1].set_title('Displaced $T_1$');

dims = np.array(bw_img.shape)
padded_img = pad_image(shift_img)

ax[2].imshow(padded_img, cmap='bone')
ax[2].set_title('Padded $T_1$');
shift_img_roll2 = np.roll(np.roll(padded_img, -4, axis=0), 4, axis=1)
ax[3].imshow(shift_img_roll2, cmap='bone')
ax[3].set_title('Displaced padded $T_1$');

```



With zero padding, we add many more pixels. In this case we added the size of the image in all directions, which result in a nine time larger image. This will of course also increase the processing time. Therefore it is common to only add as much padding as the greatest displacement we expect.

## Demonstration of the correlation in space

### 0.11.9 Other metrics

#### Mean Squared Error

We can also use

- MSE
- or RMSE

and look for minima

#### Testing MSE

```

out_score = np.zeros(nx.shape, dtype=np.float32)

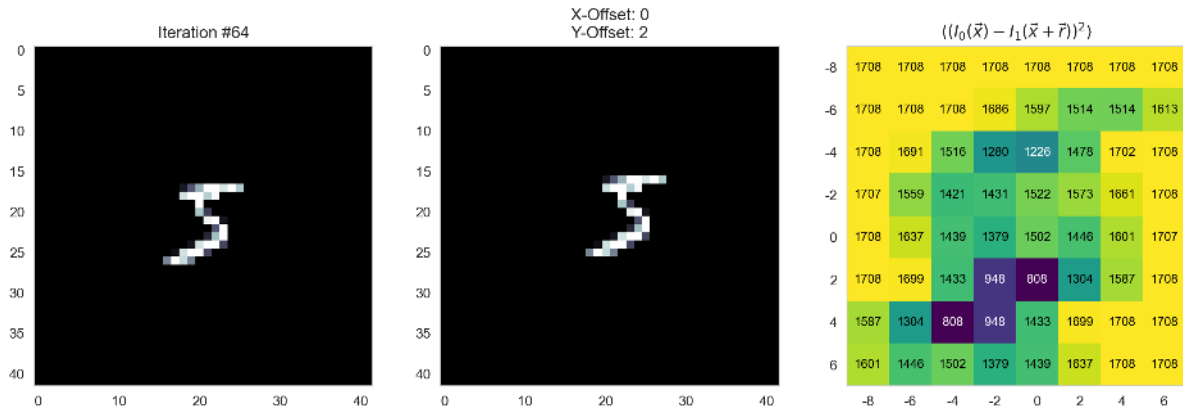
for i in range(len(nx)):
    a_img = pbw_img
    b_img = np.roll(np.roll(pshift_img, nx[i], axis=1), ny[i], axis=0)
    out_score[i] = np.mean(np.square(a_img-b_img))

```

(continues on next page)

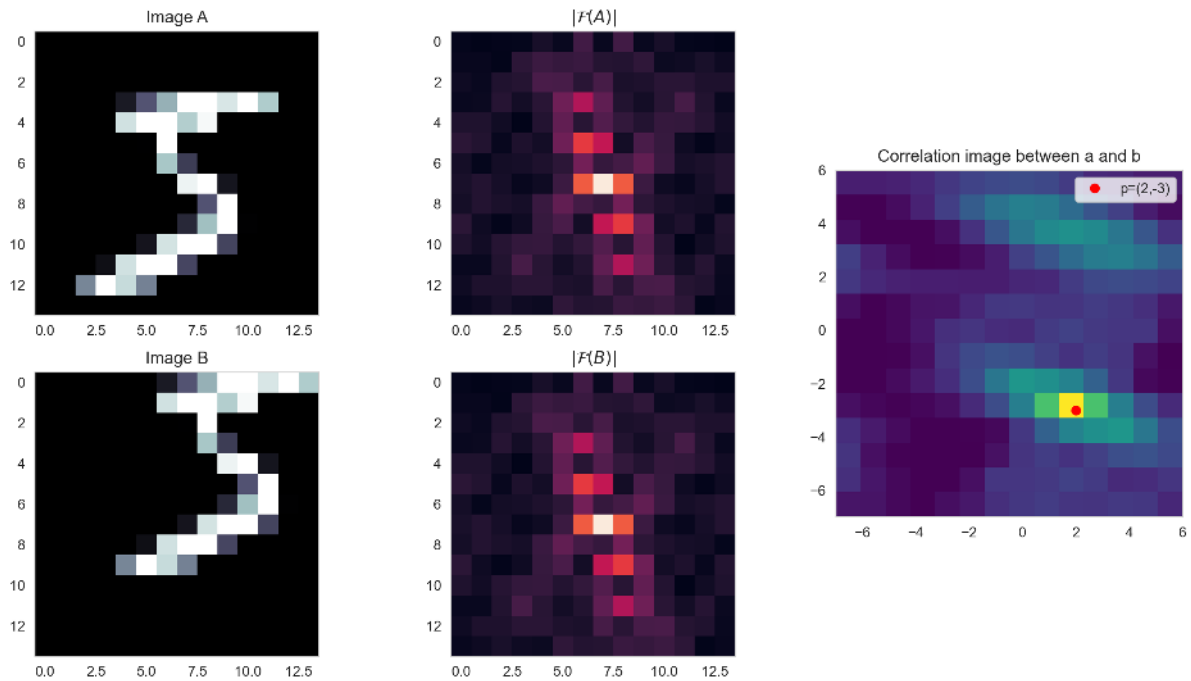
(continued from previous page)

```
# get the minimum
i_min = np.argmin(out_score)
b_img = np.roll(np.roll(pshift_img, nx[i_min], axis=1), ny[i_min], axis=0)
```



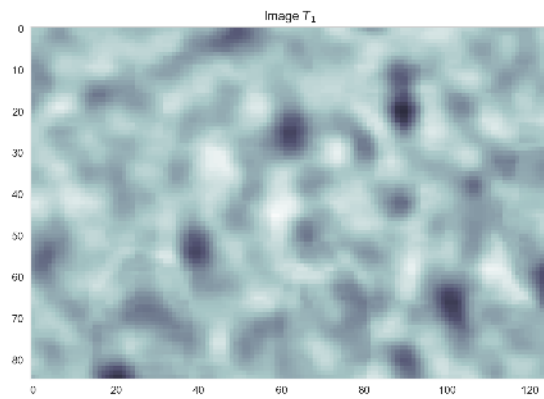
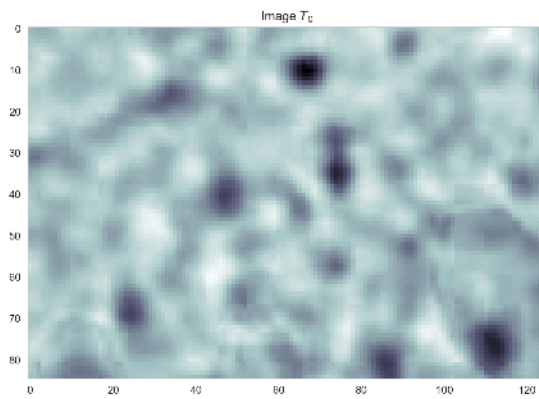
### 0.11.10 Correlation using the Fourier transform

$$C_{I_0, I_1}(\vec{r}) = \mathcal{F}^{-1}\{\mathcal{F}\{I_0\} \cdot \mathcal{F}\{I_1\}^*\}$$

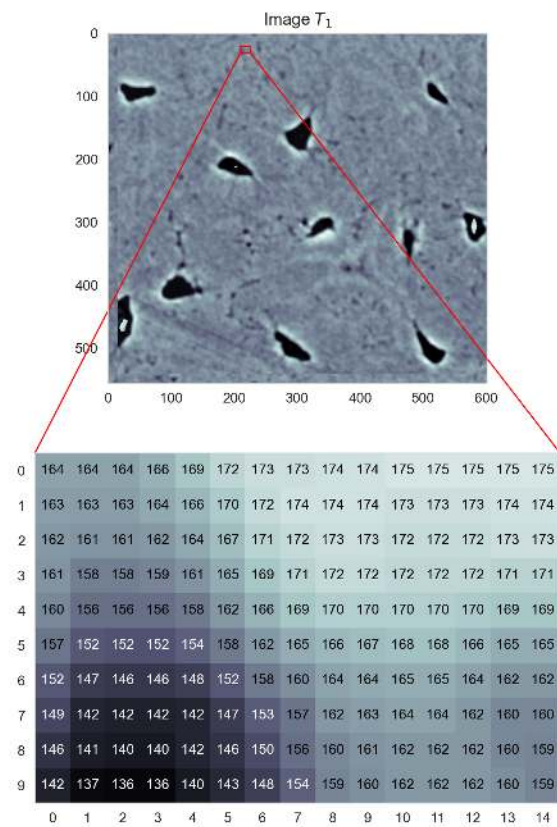
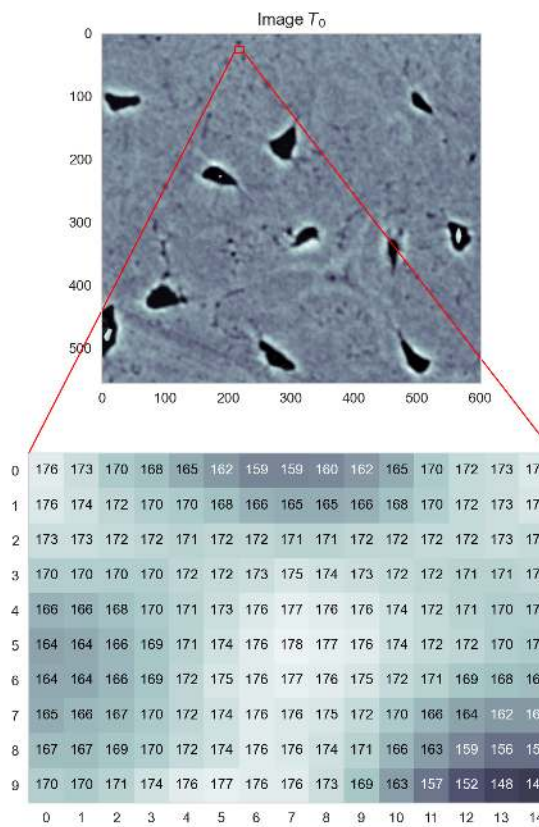


### 0.11.11 Real Examples

#### Bone Slice Registration



Let's look at a smaller region



## Computing the correlation sum for each pixel

```

from matplotlib.animation import FuncAnimation
from IPython.display import HTML
fig, (ax1, ax2, ax3) = plt.subplots(1, 3, figsize=(15,5),dpi=200)

def g_roi(x): return x[20:30:2, 210:225:2]

mx, my = np.meshgrid(np.arange(-10, 12, 4),
                     np.arange(-10, 12, 4))

nx = mx.ravel()
ny = my.ravel()
out_score = np.zeros(nx.shape, dtype=np.float32)

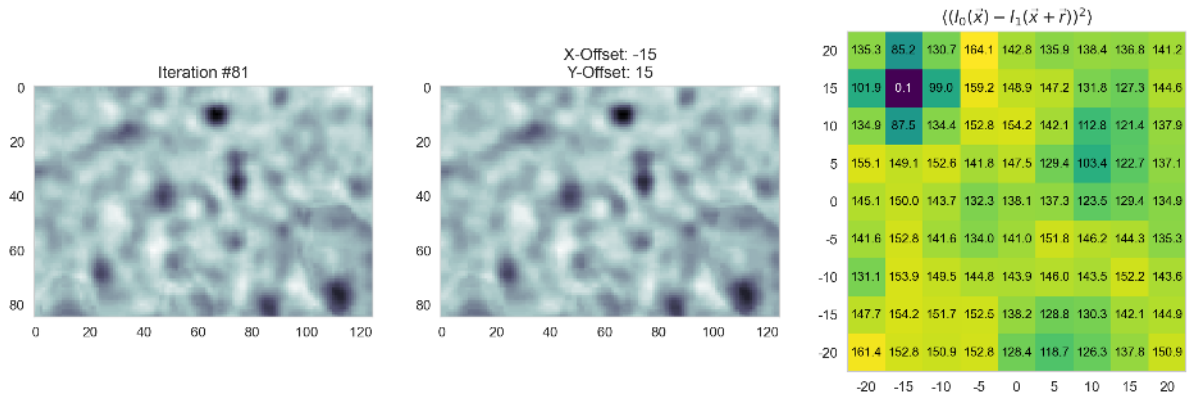
def update_frame(i):
    a_img = g_roi(full_img)
    b_img = g_roi(np.roll(np.roll(full_shift_img, nx[i], axis=1), ny[i], axis=0))
    ax1.cla(),
    # sns.heatmap(a_img, ax=ax1, cbar=False, annot=True, fmt='0.0f', cmap='bone')
    ps.heatmap(a_img, ax=ax1, bgbox=False, cmap='bone', fontsize=10, precision=0)
    ax2.cla(),
    # sns.heatmap(b_img, ax=ax2, cbar=False, annot=True, fmt='0.0f', cmap='bone')
    ps.heatmap(b_img, ax=ax2, bgbox=False, cmap='bone', fontsize=10, precision=0)
    out_score[i] = np.mean(np.square(a_img-b_img))
    ax3.cla(),
    # sns.heatmap(out_score.reshape(mx.shape), ax=ax3, cbar=False, annot=True, fmt='2.
    ↪1f', cmap='RdBu')
    ps.heatmap(out_score.reshape(mx.shape), ax=ax3, bgbox=False, cmap='RdBu', fontsize=10,
    ↪precision=1, vmin=0.0, vmax=420.0)
    ax1.set_title('Iteration #{}'.format(i+1))
    ax2.set_title('X-Offset: %d\ nY-Offset: %d' % (2*nx[i], 2*ny[i]))
    ax3.set_xticklabels(mx[0, :])
    ax3.set_yticklabels(my[:, 0])
    ax3.set_title(r'$\langle I_0(\vec{x}) - I_1(\vec{x} + \vec{r}) \rangle^2 \rangle$');

# write animation frames
anim_code = FuncAnimation(fig,
                          update_frame,
                          frames=len(nx),
                          interval=300,
                          repeat_delay=2000)
anim_code.save('movies/spacecorrelation_bone.mp4', fps=5)
plt.close('all')

```

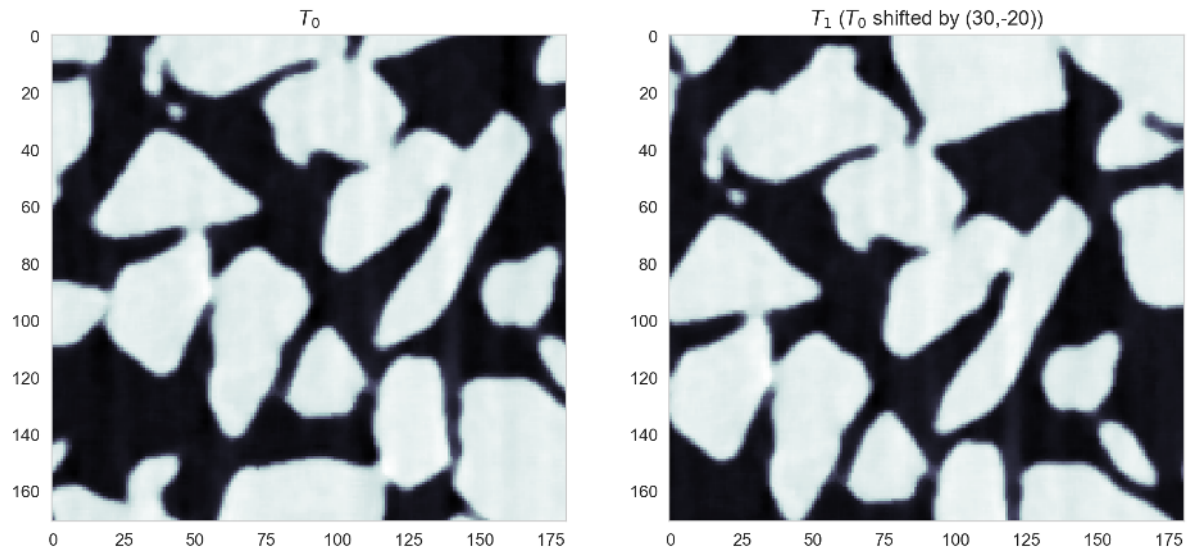
## Checking the results

The analysis resulted in a minimum at displacement  $\Delta x = -15, \Delta y = 15$



## Sand grains

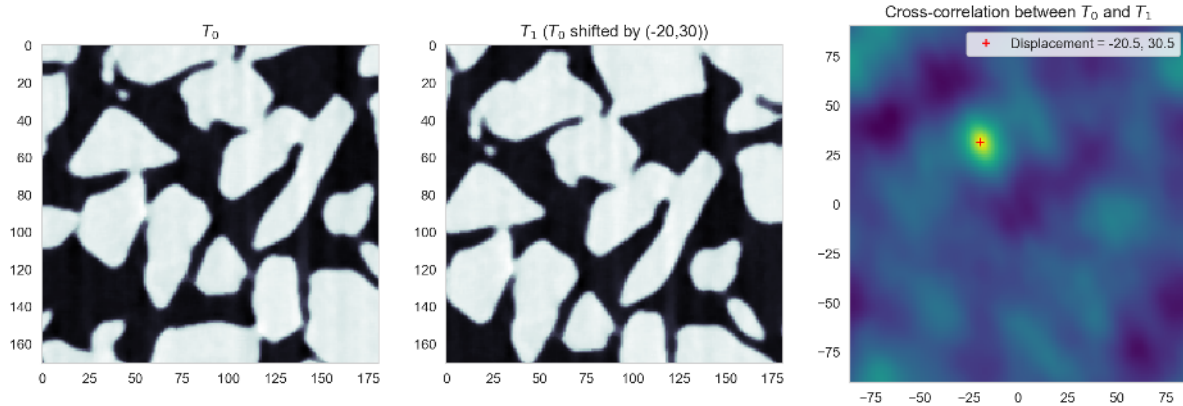
```
bw_sand = plt.imread('data/sand.png').astype(float) # Some sand grains
shift_sand = bw_sand[:-30,20:]
bw_sand = bw_sand[30:,:shift_sand.shape[1]]
```



## Correlation in Fourier space

```
fg0=np.fft.fft2(bw_sand)
fg1=np.fft.fft2(shift_sand)
fg0g1=fg1*np.conj(fg0)
cg0g1=np.real(np.fft.fftshift(np.fft.ifft2(fg0g1)))

pos=np.unravel_index(np.argmax(cg0g1[::-1]),cg0g1.shape)
x=pos[1]-cg0g1.shape[1]/2
y=-(pos[0]-cg0g1.shape[0]/2)
```



## 0.12 Registration

Before any meaningful tracking tasks can be performed, the first step is to register the measurements so they are all on the same coordinate system.

Often the registration can be done along with the tracking by separating the movement into actual sample movement and other (camera, setup, etc) if the motion of either the sample or the other components can be well modeled.

In medicine this is frequently needed because different scanners produce different kinds of outputs with different scales, positioning and resolutions. This is also useful for ‘follow-up’ scans with patients to identify how a disease has progressed. With scans like chest X-rays it isn’t uncommon to have multiple (some patients have hundreds) all taken under different conditions

### 0.12.1 The Process

We informally followed a process before when trying to match the two images together, but we want to make this more generic for a larger spectrum of problems.

We thus follow the model set forward by tools like [ITK](#) with the components divided into the input data:

- *Moving Image*
- and *Fixed Image* sometimes called *Reference Image*).

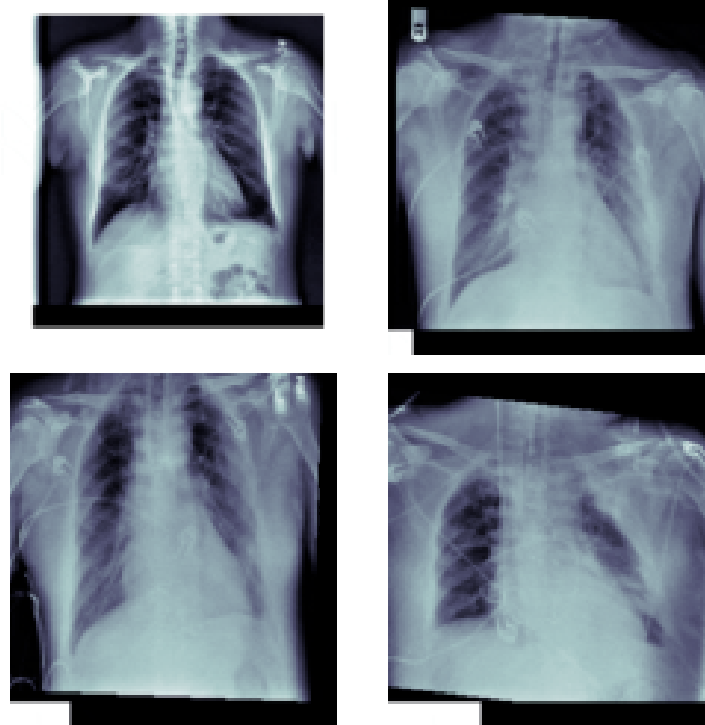


Fig. 1: A series of chest X-ray images taken at different times.

### The algorithmic components

- The *Transform* operation to transform the moving image.
- The *interpolator* to handle bringing all of the points onto a pixel grid.
- The *Metric* which is the measure of how well the transformed moving image and fixed image match
- and finally the *Optimizer* that tries to find the best solution

```
<IPython.core.display.SVG object>
```

### An automatic registration algorithm

```
<IPython.core.display.SVG object>
```

## 0.12.2 Images

### Fixed Image

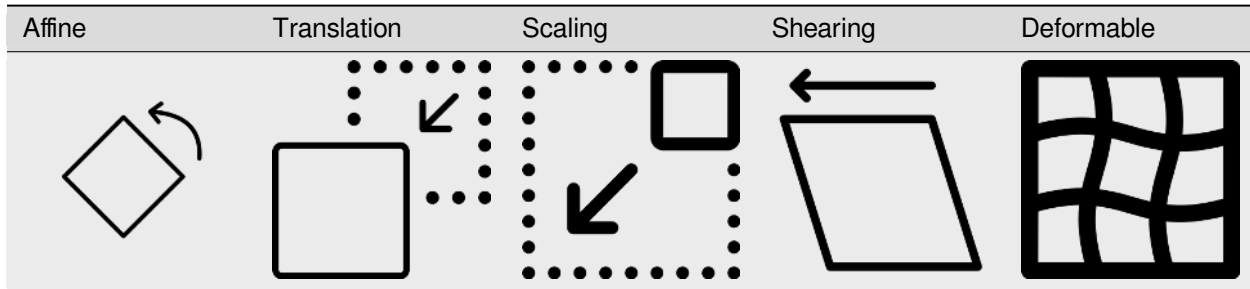
The fixed image (or reference image) is the image that will be left untouched and used for comparison

### Moving Image

The moving image will be transformed (translated, scaled, rotated, deformed, ...) to try and match as closely as possible the fixed image.

#### 0.12.3 Transform

The transform specifies the transformations which can take place on the moving image, a number of different types are possible, but the most frequent types are listed below.



#### 0.12.4 Interpolator

The interpolator is the component applies the transform to the moving image. The common ways of interpolating are

- Nearest Neighbor
- Bilinear
- Bicubic
- Bspline
- ...

#### 0.12.5 Metric

The metric is how the success of the matching of the two images is measured. The goal is to measure similarity between images.

- Mean Squared Error - the simplest metric to use just recording the raw difference, but often this can lead to unusual matches since noise and uneven illumination can lead to high MSE for images that match well.
- SSIM similarity metric
- Correlation Factor



## 0.12.6 Optimizer

The optimizer component is responsible for updating the parameters based on the metric. A standard approach with this is gradient descent where the gradient is calculated and a small step (determined by the learning rate) is taken in the direction of maximum descent.

- Gradient Descent
- Adam
- Stochastic Gradient Descent
- AdaGrad
- AdaDelta

## 0.12.7 Registration with different metrics

First we need an image...

```
# Some support functions
import skimage.transform as transform
from matplotlib.patches import Rectangle
import registration as reg

def create_demo_pair(img,dx=20,dy=-10,theta=0.0):
    # Create moving by applying a known translation
    true_shift = (-dx, -dy, theta) # tx, ty

    fixed0 = img[200:600,200:600]

    moving0 = transform.rotate(img,theta)[(200+dy):(600+dy),(200+dx):(600+dx)]

    fixed = reg.to_sitk(fixed0)
    moving = reg.to_sitk(moving0)

    return fixed,moving, true_shift

def show_reg_result(fixed,moving,res):
    fig,ax = plt.subplots(1,4,figsize=(15,3.4))

    h = res["history"]
    tx = np.array(h["tx"], dtype=float)
    ty = np.array(h["ty"], dtype=float)
    ax[0].plot(tx,ty,marker='.')
    ax[0].set(title='Trajectory',xlabel='x',ylabel='y')

    dist = np.sqrt((tx-true_shift[0])**2+(ty-true_shift[1])**2)
    ax[1].plot(dist)
    ax[1].set(title='Distance to target',xlabel='iterations',ylabel='distance')

    reg_np = reg.image_to_np(res["registered"])
    ax[2].imshow(reg_np,cmap='gray')
    ax[2].set(title='Registered')

    ax[3].imshow(reg.image_to_np(fixed)-reg_np,vmin=-0.01,vmax=0.01,cmap='coolwarm')
    ax[3].set(title='Original-Registered')
```

```
img = plt.imread('data/NMC_90wt_2000bar_115.tif')[:, :, 0].astype('float')
fixed, moving, true_shift = create_demo_pair(img)
```

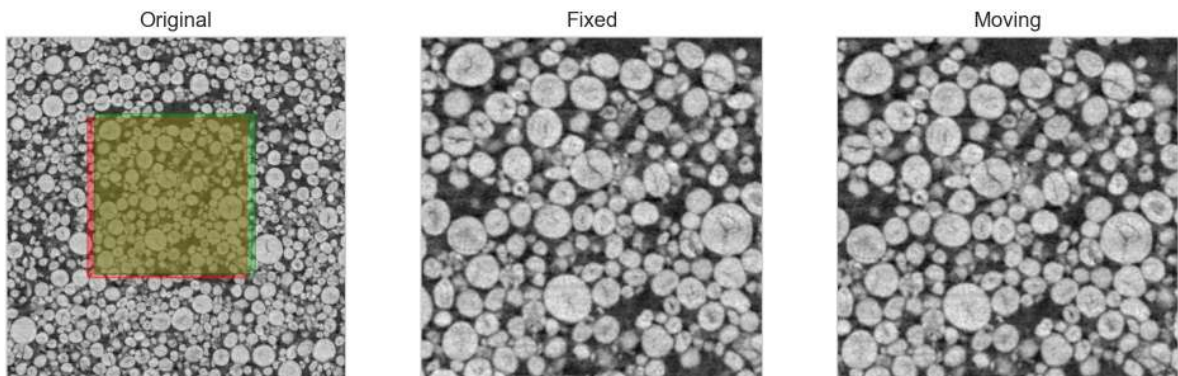
```
fig, ax = plt.subplots(1, 3, figsize=(12, 3.5))

ax[0].imshow(img, cmap='gray')
ax[0].set(title="Original", xticks=[], yticks=[])
rect = Rectangle(
    xy=(200, 200),      # (x, y) of lower-left corner
    width=400,
    height=400,
    linewidth=1,
    edgecolor='red',
    facecolor='red',
    alpha=0.35,
)
ax[0].add_patch(rect)

rect = Rectangle(
    xy=(220, 190),      # (x, y) of lower-left corner
    width=400,
    height=400,
    linewidth=1,
    edgecolor='darkgreen',
    facecolor='limegreen',
    alpha=0.35,
)
ax[0].add_patch(rect)

ax[1].imshow(reg.image_to_np(fixed), cmap='gray')
ax[1].set(title="Fixed", xticks=[], yticks=[])

ax[2].imshow(reg.image_to_np(moving), cmap='gray')
ax[2].set(title="Moving", xticks=[], yticks=[]);
```



The image moving image is displaced by  $dx=20$  and  $dy=-10$

## Our workflow

This example uses SimpleITK. It requires quite much preparation

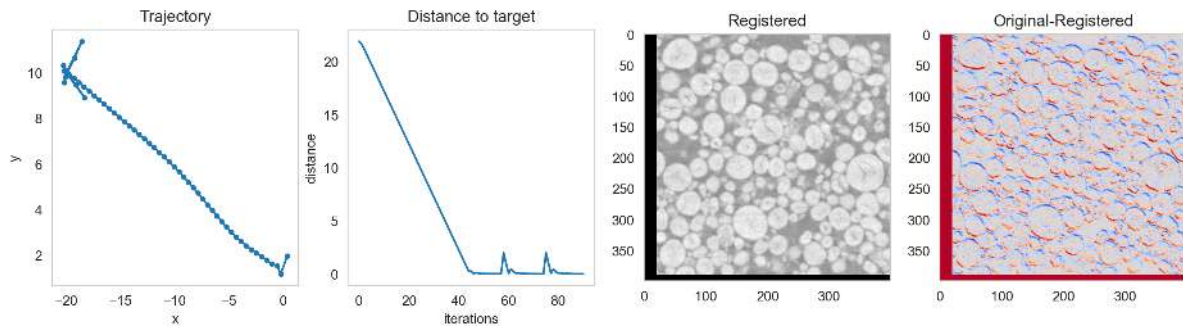
- Define a registration model
- Add transformer
- Add metric
- Add optimizer

This includes quite many lines of which are placed in a separate file `registration.py`

### 0.12.8 Registration with MSE metric

The mean squared error is a intuitive first metric  $MSE = \frac{1}{N} \sum_{p=1}^N (a(p) - b(p))^2$

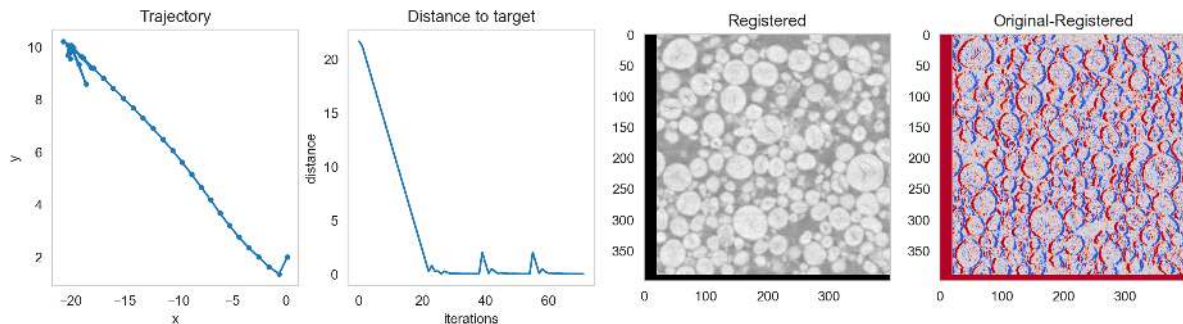
```
res = reg.run_registration(fixed, moving, "MeanSquares")
show_reg_result(fixed, moving, res)
```



### 0.12.9 Registration with correlation metric

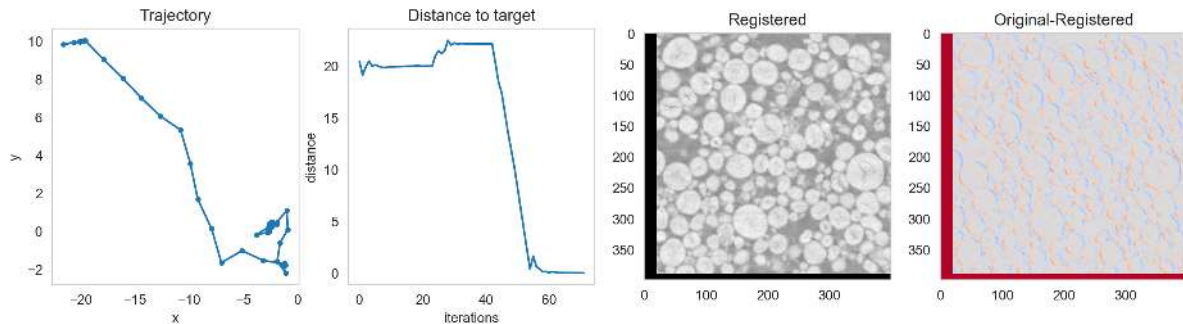
The correlation check how similar the images are  $Corr = \frac{\sum_{i=1}^N x_i \cdot y_i}{\sum_{i=1}^N x_i^2 \sum_{i=1}^N y_i^2}$

```
res = reg.run_registration(fixed, moving, "Correlation")
show_reg_result(fixed, moving, res)
```



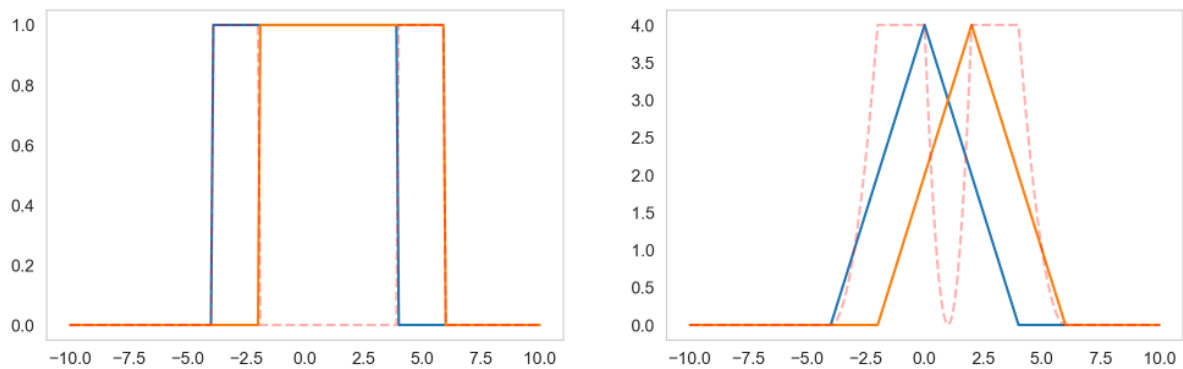
### 0.12.10 Registration with joint histogram mutual information metric

```
res = reg.run_registration(fixed, moving, "JointHistogramMI")
show_reg_result(fixed, moving, res)
```



### 0.12.11 Image characteristics and registration

Registration is gradient based



### 0.12.12 Smoother Gradient

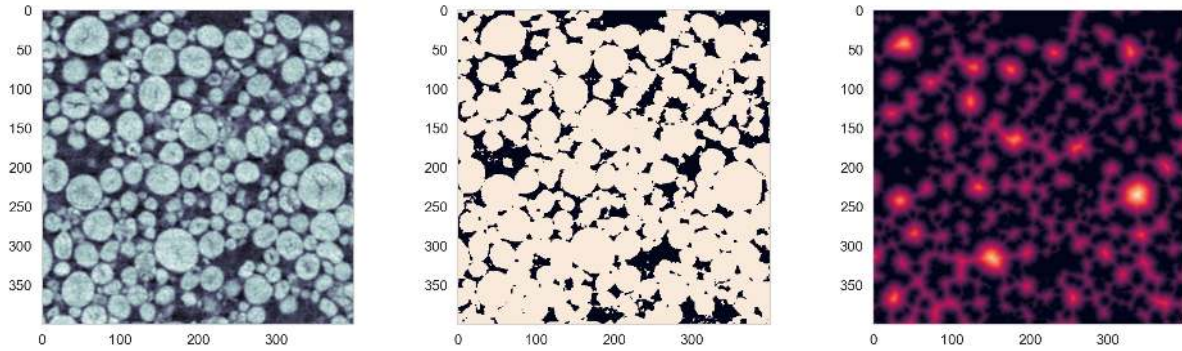
We can use a distance map of the segmentation to give us a smoother gradient

#### Prepare images

The image preparation this time requires a bit more work. To obtain the distance map, we have to

1. Threshold the image. Here, I used a threshold at intensity 50.
2. Compute the distance map.

Afterwards we use the same steps as with the previous image.



### The resulting registration using a distance map

```

-----
NameError                                Traceback (most recent call last)
Cell In[69], line 1
----> 1 show_registration(dfixed_image,dmoving_image,dframes,dpars,dlosses)

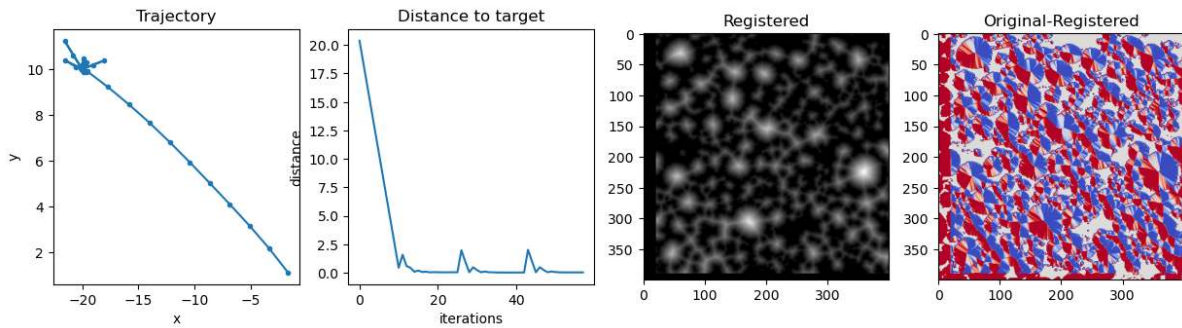
NameError: name 'show_registration' is not defined

```

```

res = reg.run_registration(reg.to_sitk(a), reg.to_sitk(b), "MeanSquares")
show_reg_result(reg.to_sitk(a), reg.to_sitk(b), res)

```



We see here that registration using a distance map produces an almost straight track, while the registration with the gray level image had to search a bit before finding the optimal registration.

### Compare losses

The straighter registration track also results in faster convergence as we can see when the two losses are plotted together. Here, we look at the normalized losses because the MSE is much greater in the gray level images due to a different gray level distribution compared to the distance map.

This observation tells us that the choice of input images to the registration algorithm has an impact on the result. Both in terms of precision and in convergence. High contrast images are easier to register. The trick of using the distance map can actually improve the results if you have low contrast images.

### 0.12.13 A note on initial guess

The registration is, similar to the tracking, sensitive to great distances and rotations.

In particular when:

- There are few characteristic features in the images
- Noisy images

A good initial guess can improve the registration performance.

### 0.12.14 Local changes between images

We can approach the problem by subdividing the data into smaller blocks and then apply the digital volume correlation independently to each block.

- information on changes in different regions
- less statistics than a larger box

## 0.13 Introducing Physics

DIC or DVC by themselves include no *sanity check* for realistic offsets in the correlation itself.

The method can, however be integrated with physical models to find a more optimal solutions.

- information from surrounding points
- smoothness criteria
- maximum deformation / force
- material properties

$$C_{\text{cost}} = \underbrace{C_{I_0, I_1}(\vec{r})}_{\text{Correlation Term}} + \underbrace{\lambda ||\vec{r}||}_{\text{deformation term}}$$

### 0.13.1 Distribution Metrics

As we covered before distribution metrics like the distribution tensor can be used for tracking changes inside a sample.

Of these the most relevant is the texture tensor from cellular materials and liquid foam. The texture tensor is the same as the distribution tensor except that the edges (or faces) represent physically connected / touching objects rather than touching Voronoi faces (or conversely Delaunay triangles).

These metrics can also be used for tracking the behavior of a system without tracking the single points since most deformations of a system also deform the distribution tensor and can thus be extracted by comparing the distribution tensor at different time steps.

### 0.13.2 Quantifying Deformation: Strain

We can take any of these approaches and quantify the deformation using a tool called the strain tensor.

Strain is defined in mechanics for the simple 1D case as the change in the length against the change in the original length.

$$e = \frac{\Delta L}{L}$$

While this defines the 1D case well, it is difficult to apply such metrics to voxel, shape, and tensor data.

### 0.13.3 Strain Tensor

There are a number of different ways to calculate strain and the strain tensor, but the most applicable for general image based applications is called the **infinitesimal strain tensor**, because the element matches well to square pixels and cubic voxels.

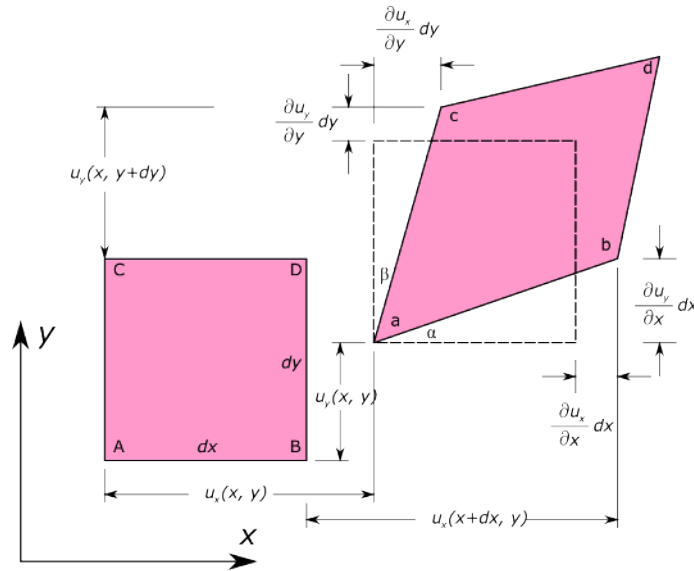


Fig. 1: Deformation by strain.

### 0.13.4 Types of Strain

We categorize the types of strain into two main categories:

$$\underbrace{\mathbf{E}}_{\text{Total Strain}} = \underbrace{\varepsilon_M \mathbf{I}_3}_{\text{Volumetric}} + \underbrace{\mathbf{E}'}_{\text{Deviatoric}}$$



### Volumetric / Dilational

The isotropic change in size or scale of the object.

### Deviatoric

The change in the proportions of the object (similar to anisotropy) independent of the final scale

### Shear band

S. Hall 2013

## 0.14 Two Point Correlation - Volcanic Rock

### Data provided by Mattia Pistone and Julie Fife

The air phase changes from small very anisotropic bubbles to one large connected pore network.

- The same tools cannot be used to quantify those systems.
- Furthermore there are motion artifacts which are difficult to correct.

We can utilize the two point correlation function of the material to characterize the shape generically for each time step and then compare.

## 0.15 Summary

### 0.15.1 Time series analysis

- Dynamic experiments
- Object tracking
- Registration
- Digital volume correlation (DIC)

## 0.16 Visualization

### 0.16.1 Load needed modules

```
import matplotlib.pyplot as plt
import seaborn as sns
from skimage.morphology import label
from skimage.measure import regionprops
import pandas as pd

from skimage.feature import corner_peaks, corner_harris, BRIEF
from skimage.transform import warp, AffineTransform
```

(continues on next page)



(continued from previous page)

```

from skimage import data
from skimage.io import imread
from scipy.ndimage import distance_transform_edt
from skimage.filters import threshold_otsu

from matplotlib.patches import Rectangle
from matplotlib.patches import ConnectionPatch

import numpy as np
import pydot

%load_ext autoreload
%autoreload 2
plt.rcParams['font.family'] = ['sans-serif']
plt.rcParams['font.sans-serif'] = ['DejaVu Sans']
plt.style.use('default')
sns.set_style("whitegrid", {'axes.grid': False})

%config InlineBackend.figure_format = 'retina'

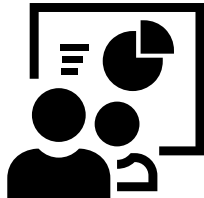
```

## 0.17 Presenting the results - bringing out the message

In the end you will want to present your results



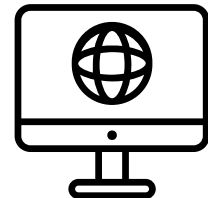
Discussions



Presentation



Publication



Web page

Fig. 1: Different ways to present your data.

### 0.17.1 Visualization

One of the biggest problems with *big* sciences is trying to visualize a lot of heterogeneous data.

- Tables are difficult to interpret
- 3D Visualizations are very difficult to compare visually
- Contradictory necessity of simple single value results and all of the data to look for trends and find problems

## Purpose of the visualization

You visualize your data for different reasons:

1. **Understanding and exploration**

- Small and known audience (you and colleagues)
- High degree of understanding of specific topic.

### 2. Presenting your results

- Wider and sometimes unknown audience (reader of paper, person listening to presentation)
- At best general understanding of the topic.

from [Knafllic 2015](#)

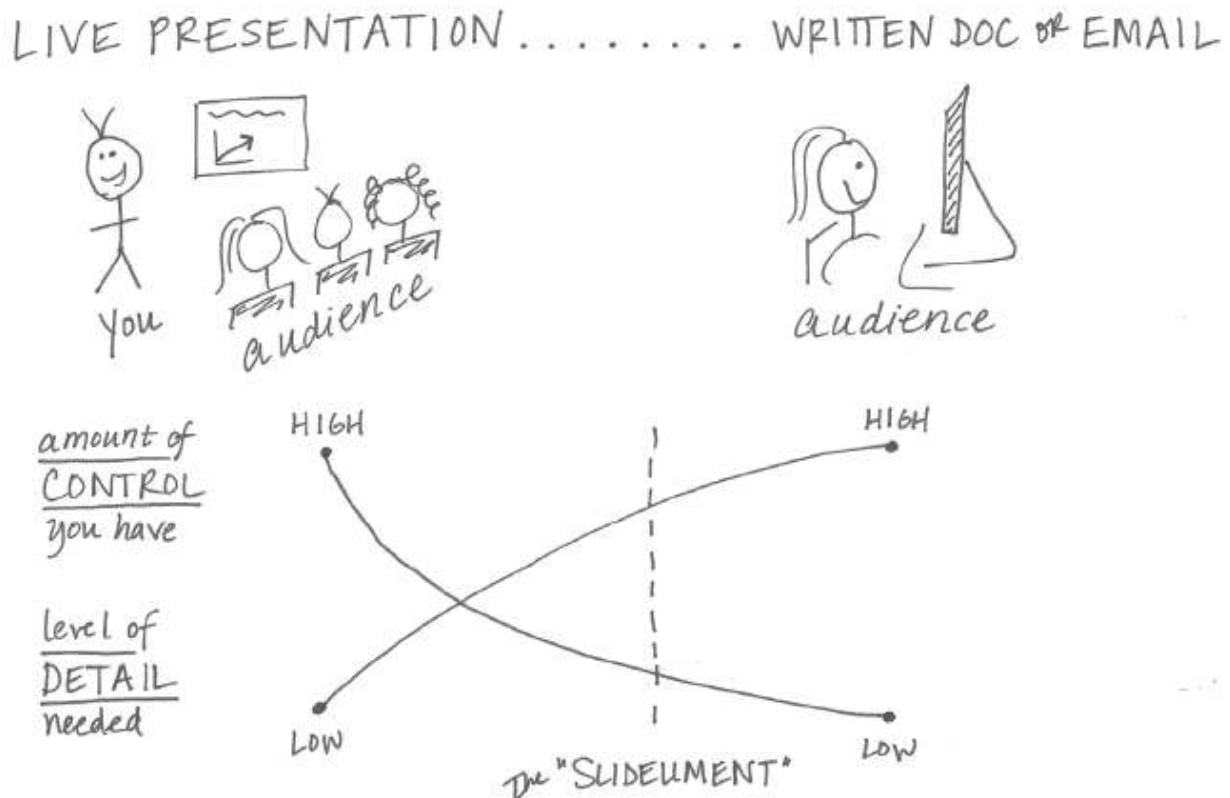


Fig. 2: The level of detail in a presentation depends on the medium it is presented [Knafllic 2015](#).

### 0.17.2 Bad Graphs

There are too many graphs which say:

- *my data is very complicated*
- *I know how to use \_\_ toolbox in Python/Matlab/R/Mathematica*
- Most programs by default make poor plots
- Good visualizations takes time to produce

[xkcd](#)

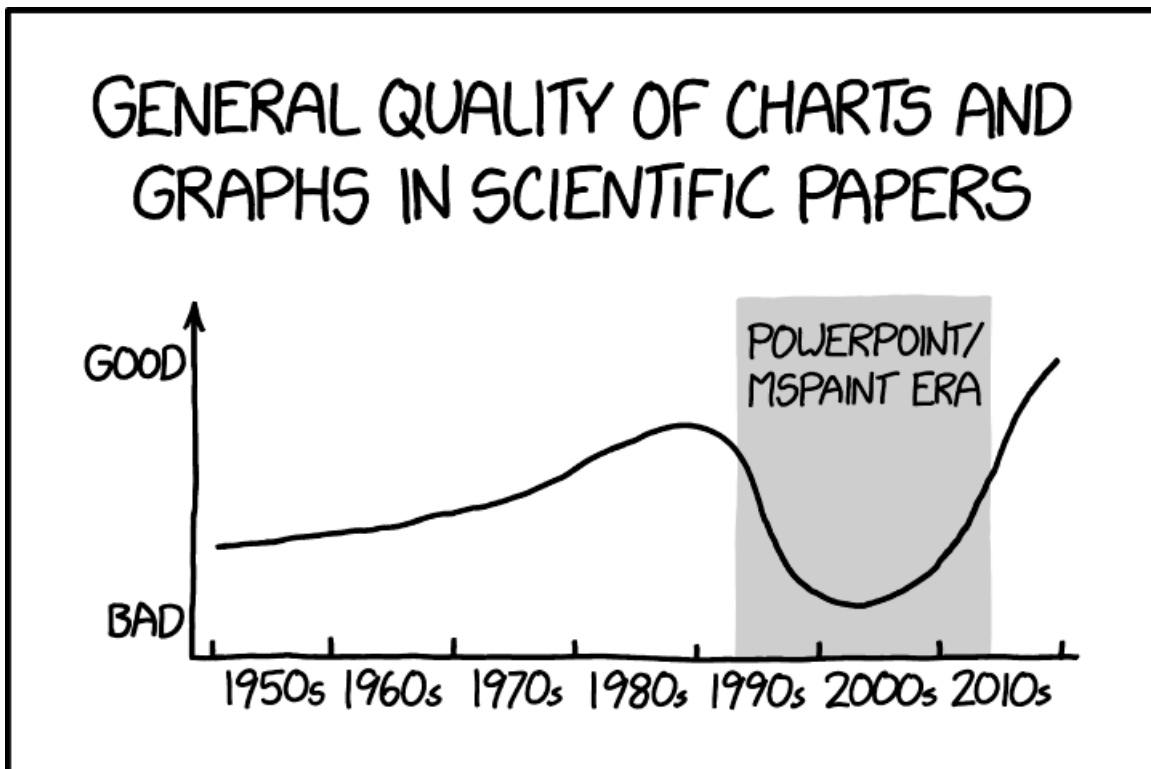


Fig. 3: This cartoon from [XKCD](#) highlights a problem with the access to software making it too easy to produce a graph or illustration. Unfortunately, without any thoughts about information content and artistic or illustration rules.

### Some bad examples

There are plenty examples on how you shouldn't present your data. The problem is in general that there is way too much information that needs to be predigestend before it is ready to any audience.

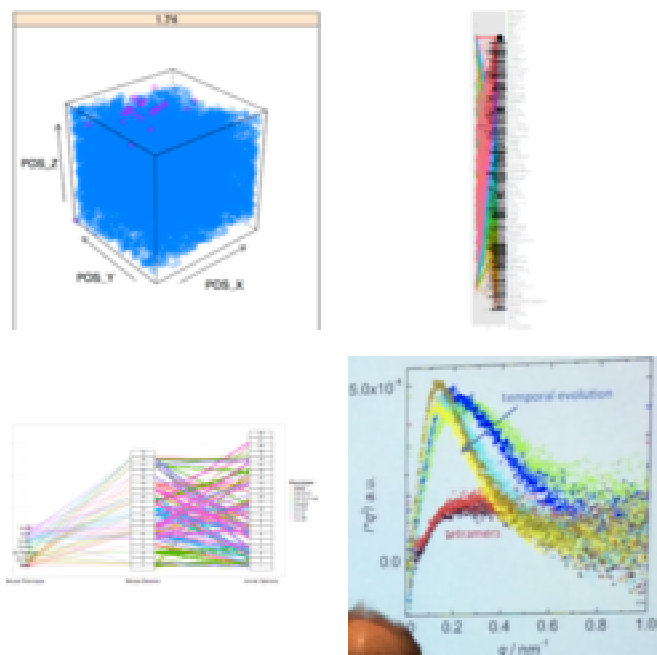


Fig. 4: Four examples of how *not* to present your data.

### 0.17.3 How to improve - Key Ideas

There is a need to consciously prepare your figures to bring your message to the audience in an understandable way. The first step is to ask yourself the following questions.

What is my message?

Is it clearly communicated?

Is it really necessary?

- Does every line / color serve a purpose?
- Pretend ink is very expensive

Keep this in mind every time you create a figure and you will notice that after while you will have a tool set that makes it easier and faster to produce well thought figures that clearly brings out you message to your audience.

Personally, I always write scripts to produce each plot of a publication. This makes it easier to revise the manuscript in a reproducible and efficient manner. The first implementation may take longer, but the revision is done in no time.

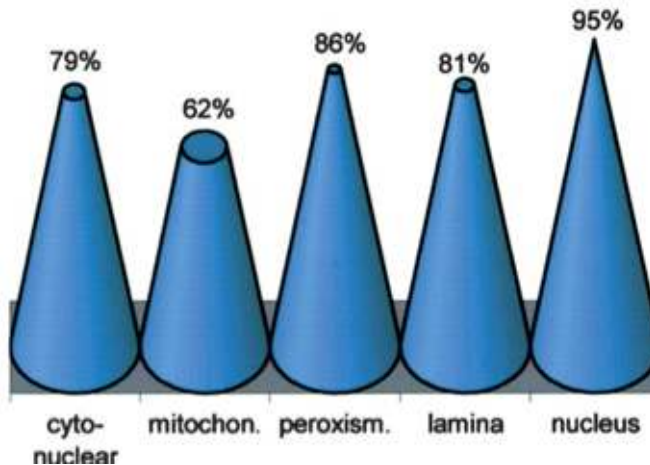
## Some literature

These are some recommended books about data visualization:

- Knaflitz, *Storytelling with Data: A Data Visualization Guide for Business Professionals*, 2015
- Few, *Should data visualization always be beautiful?*, 2012

## Simple rules

1. Never use 3D graphics when it can be avoided (unless you want to be deliberately misleading)



2. Pie charts can also be hard to interpret
3. Background color should almost always be white (not light gray)
4. Use color palettes adapted to human visual sensitivity
5. Use colors and transparency smart

## 0.17.4 Grammar of Graphics

### What is a grammar?

- Set of rules for constructing and validating a sentence
- Specifies the relationship and order between the words constituting the sentence

### How does grammar apply to graphics?

If we develop a consistent way of

- expressing graphics (sentences)
- in terms of elements (words) we can compose and decompose graphics easily

The most important modern work in graphical grammars is

“*The Grammar of Graphics*” by Wilkinson, Anand, and Grossman (2005).

This work built on earlier work by Bertin (1983) and proposed a grammar that can be used to describe and construct a wide range of statistical graphics.

### Grammar Explained

Normally we think of plots in terms of some sort of data which is fed into a plot command that produces a picture

- In Excel you select a range and plot-type and click “Make”
  - In Matlab you run `plot(xdata,ydata,color/shape)`
1. These produces entire graphics (sentences) or at least phrases in one go and thus abstract away from the idea of grammar.
  2. If you spoke by finding entire sentences in a book it would be very ineffective, it is much better to build up word by word

### Grammar

#### Separate the graph into its component parts

$$\begin{cases} var1 \rightarrow x \\ var2 \rightarrow y \end{cases}$$

Construct graphics by focusing on each portion independently.

### Figure decorations

Besides the data you also need to provide annotating items to the visualization.

It may seem unnecessary to list these annotations, but it happens too often that they are missing. This leaves the observers wondering about what they see in the figure. It is true that it takes a little more time to add annotation to your figure. Sometimes, you may think that the plot is only for your own understanding and you don't need to waste the time on making it complete. Still, in the next moment it finds its way to the presentation and then all of a sudden it is official... Annotations are fundamental features of figures and available in any plotting library. In some cases you have to look a little longer to find them or write a little more code to use them, but they are there.

### Plots

- Curve legend - telling what each curve represents.
- Axis labels - telling what information you see on each axis.
- Figure title - if you use multiples plots in the same figure.

### Images

- Color bar - to tell how the colors are mapped to the values.
- Scale bar - to tell the size of the object in the image.

## Color maps revisited

Choosing the right color is a science.

Crameri, F., et al. (2020)

The choice depends on the type of data you want to present and how humans perceive different colors. Some combinations make it easier to highlight relevant features in the images. Still, you have to be cautious not to put too much a priori information into the color map.

Visualization toolboxes provide a great collection of colormaps as we have seen several times already in this course. There are however cases when you have to define your own color map. An example is the colormap we created last week to be able to identify each item in a watershed segmented image.

### 0.17.5 What is my message?

Plots to “show the results” or “get a feeling” are usually not good

```
from plotnine import *
from plotnine.data import *
import pandas as pd
import numpy as np
# Some data
xd = np.random.rand(80)
yd = xd + np.random.rand(80)
zd = np.random.rand(80)

df = pd.DataFrame(dict(x=xd, y=yd, z=zd))
ggplot(df, aes(x='x', y='y')) + geom_point()
```

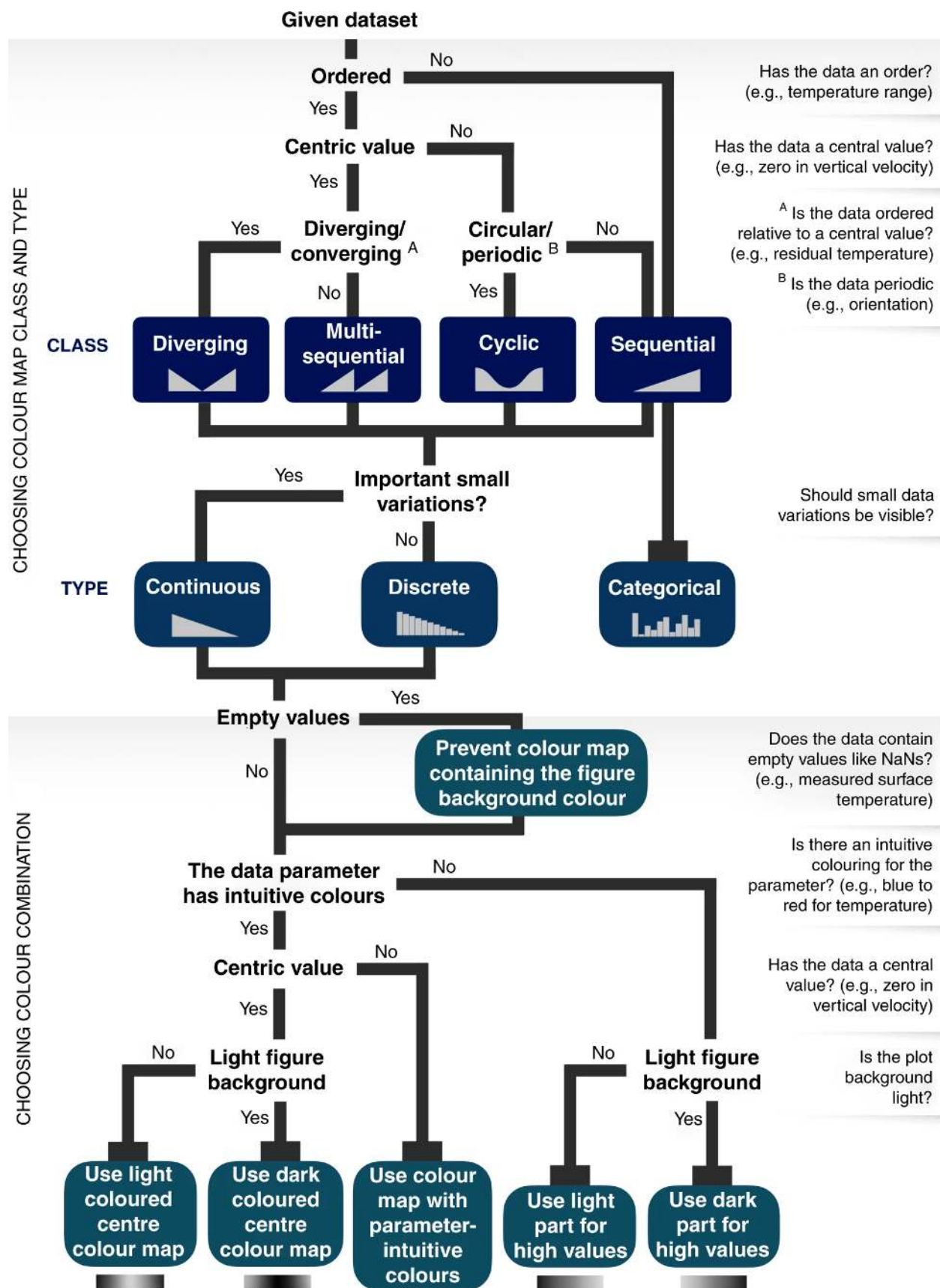
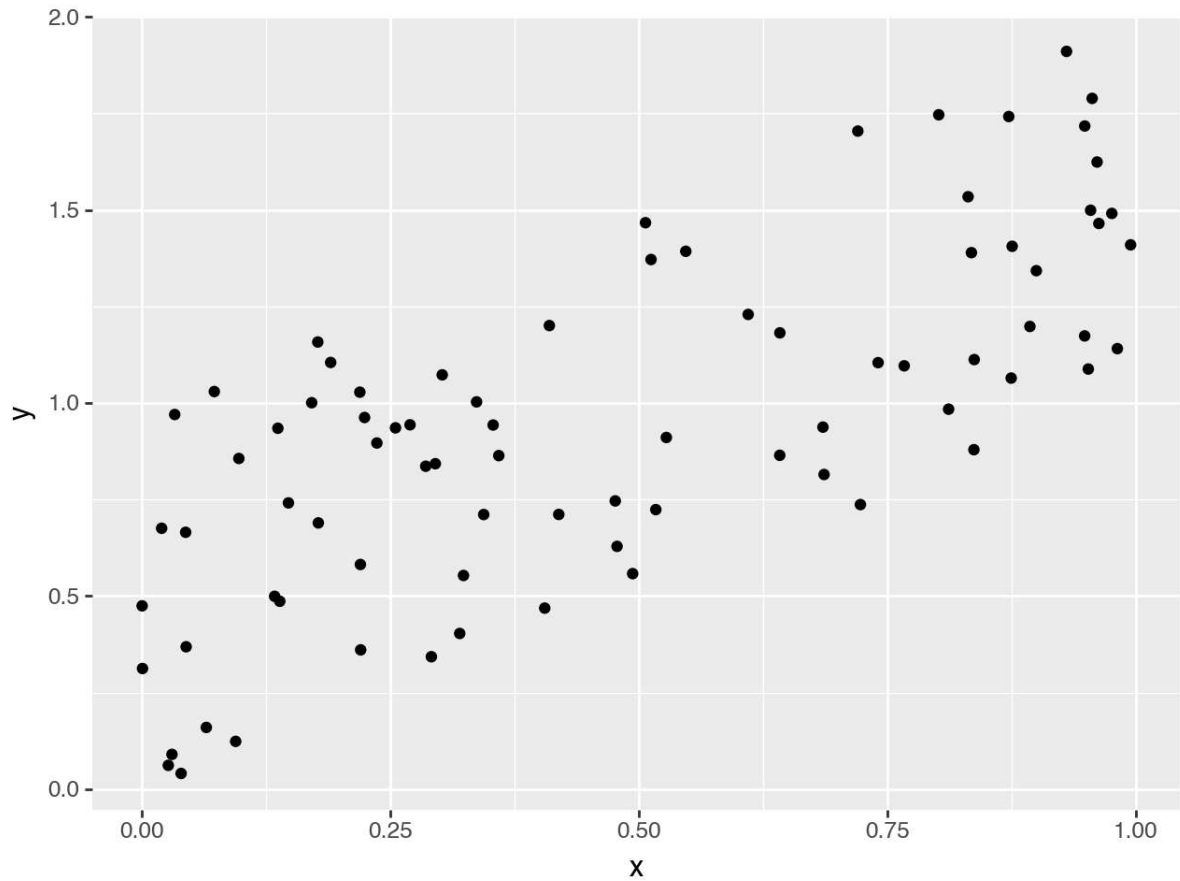


Fig. 5: Crameri et al. developed this decision flow chart to help you decide which type of color map is best suited for your data.

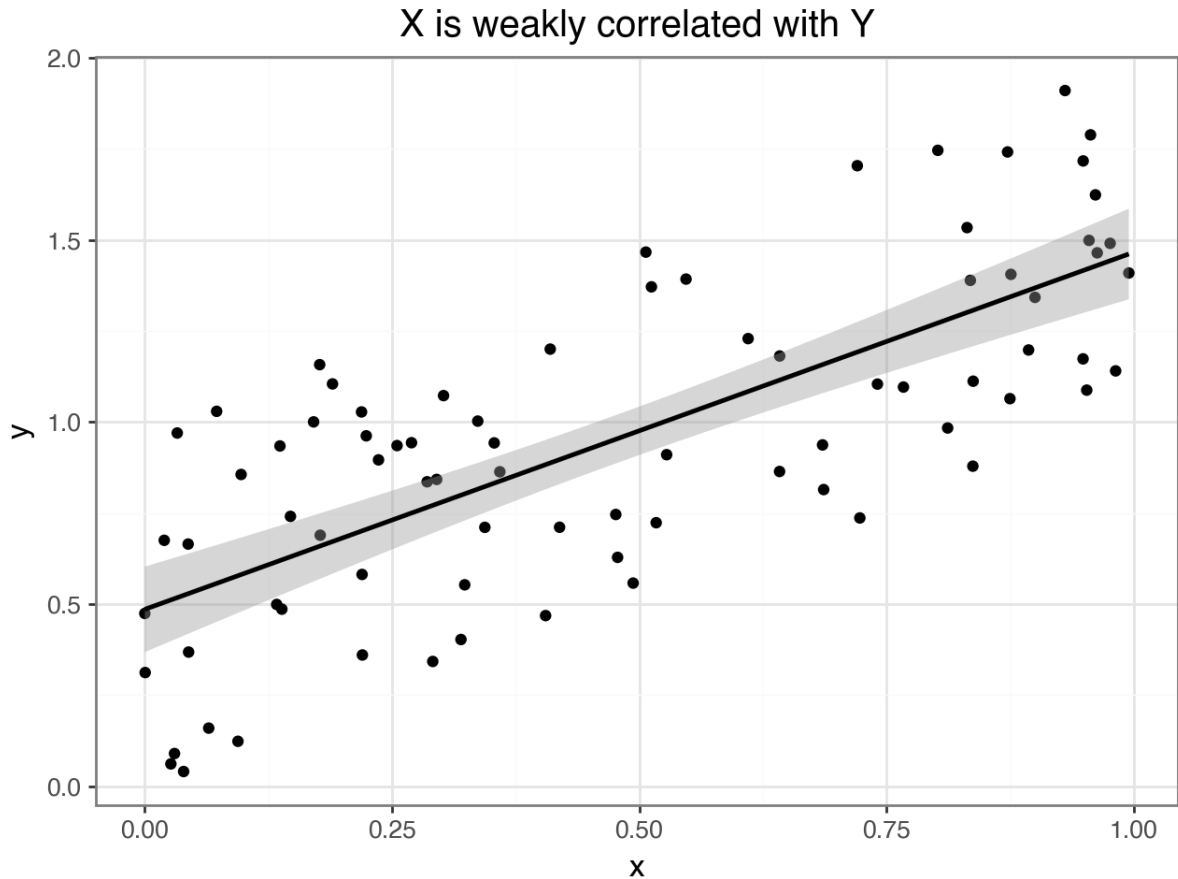




### Focus on a single, simple message

“X is a little bit correlated with Y”

```
(ggplot(df, aes(x='x', y='y'))  
  + geom_point()  
  + geom_smooth(method="lm")  
  # + coord_equal()  
  + labs(title="X is weakly correlated with Y")  
  + theme_bw(12) )
```

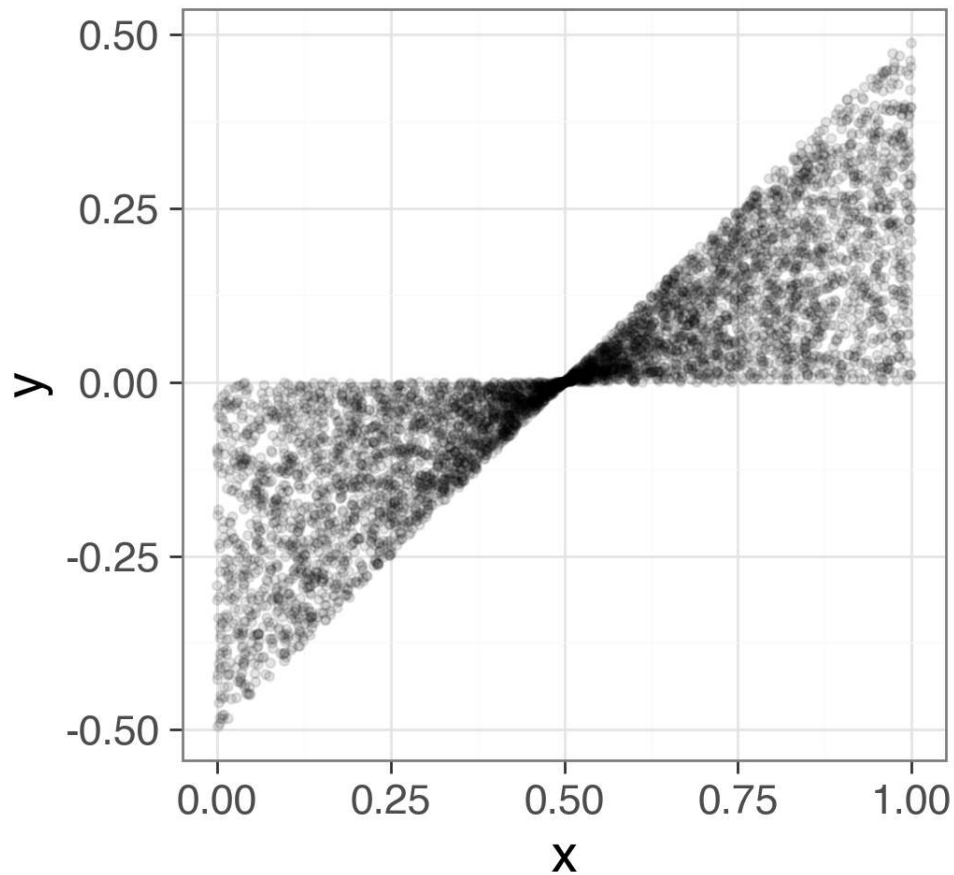


### Does my graphic communicate it clearly?

Too much data makes it very difficult to derive a clear message

```
xd = np.random.rand(5000)
yd = (xd-0.5)*np.random.rand(5000)

df = pd.DataFrame(dict(x=xd,y=yd))
(ggplot(df,aes(x='x',y='y'))
# + geom_point()
+ geom_point(alpha = 0.1)
+ coord_equal()
+ theme_bw(20))
```



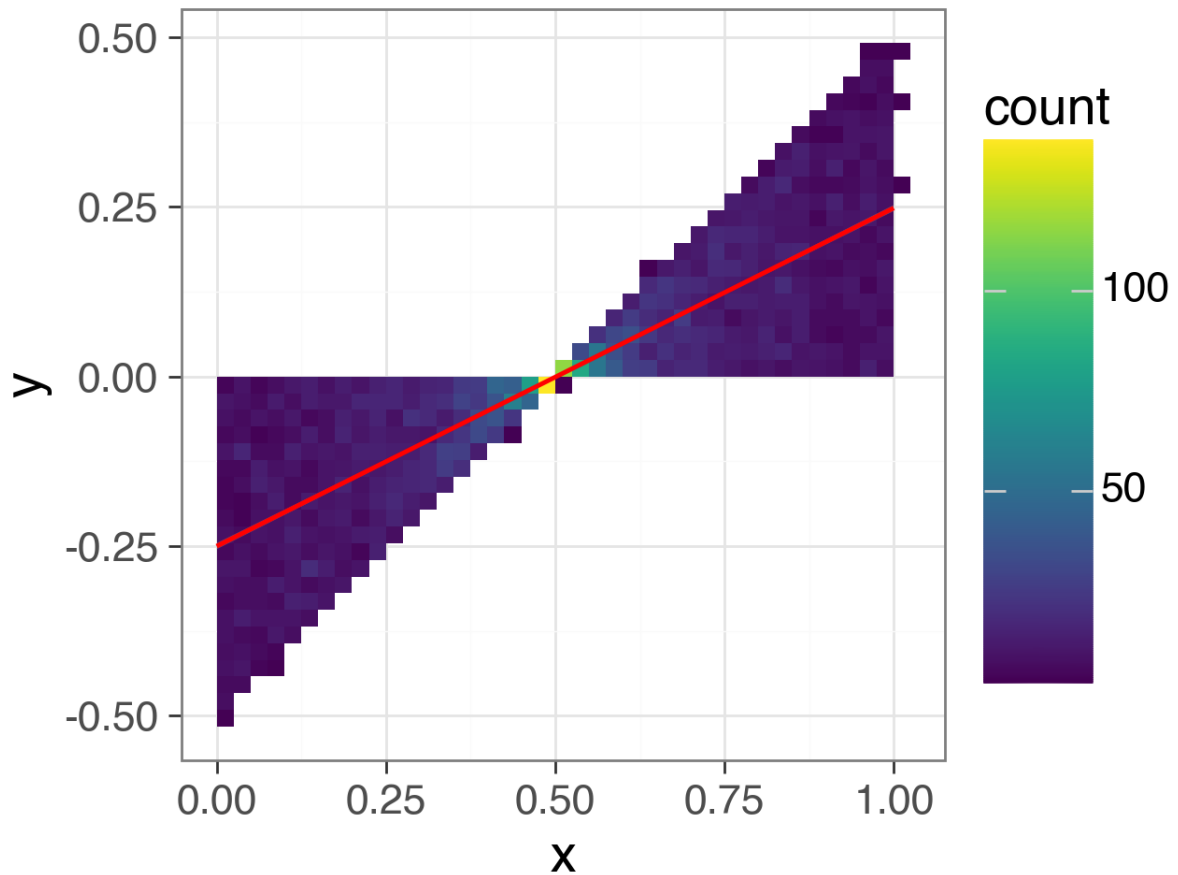
We have earlier used transparency to better visualize dense scatter plots. You can see the effect by setting the alpha parameter to `geom_point`. Using transparency is a qualitative way of showing higher density in the data.

### Reduce the data

Filter and reduce information until it is extremely simple

In this plot we create a density count view of the data by downsampling the grid and count the amount of points in each bin. It is related to a histogram but it count in space instead of in the intensity levels.

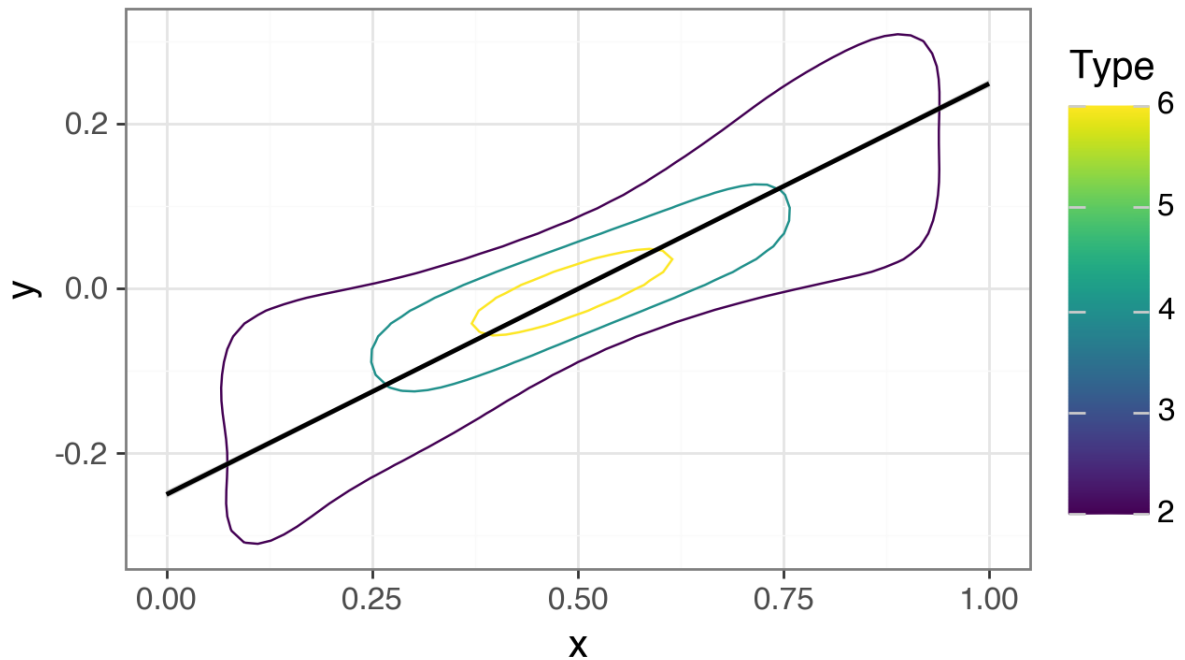
```
(ggplot(df, aes(x='x', y='y'))  
+ stat_bin_2d(bins=40)  
+ geom_smooth(method="lm", color='red')  
+ coord_equal()  
+ theme_bw(20)  
+ guides(color='F')  
)
```



Using this kind of plot allows us to measure how many points there are in each bin and thus we are now going towards a quantitative plot. The colorbar on the side helps us to interpret the colors.

### Reduce even further

```
(ggplot(df, aes(x='x', y='y'))  
  + geom_density_2d(aes(x='x', y='y', color='..level..'))  
  + geom_smooth(method="lm")  
  + coord_equal()  
  + labs(color="Type")  
  + theme_bw(15)  
)
```



### 0.17.6 Common visualization packages for python

- Matplotlib [Matplotlib 3.0 Cookbook](#) or [ETHZ lib](#), code examples
- Plotly
- Seaborn
- ggplot [R](#) using the [ggplot2](#) library, which is ported to python.

A short summary of these packages can be found [here](#).

## 0.18 Summary

### 0.18.1 Time series analysis

- Dynamic experiments
- Object tracking
- Registration
- Digital volume correlation (DIC)

### **0.18.2 Visualization**

- Visualization is the crowning piece of your investigation - make it count!
- Many toolboxes can be used, choose the one that fits your needs.